Bazele inteligenței artificiale

Laborator – Grigore Ionuț

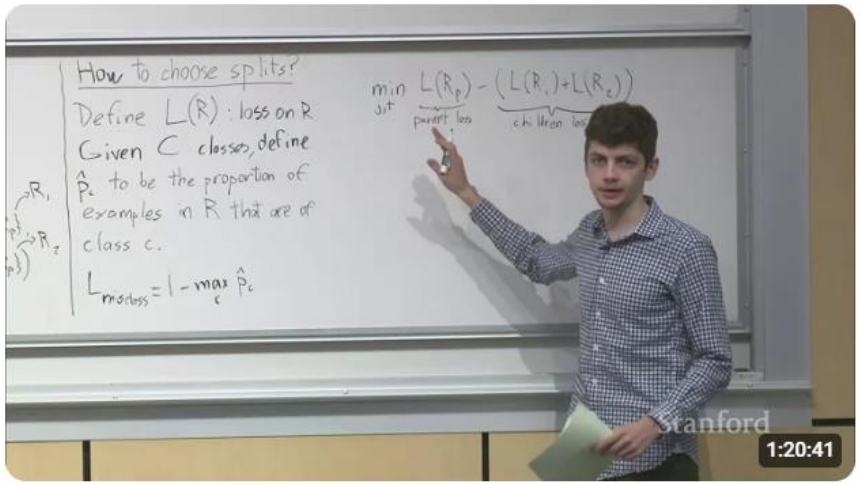
Laboratory contents

1. Introduction to Python
2. Linear models
3. Nonparametric models
4. Decision trees. Ensemble methods → today's lab
5. Dimensionality reduction
6. Clustering
7. Test 1
8. Introduction to PyTorch
9. Multilayer perceptrons
10. Convolutional neural networks
11. Recurrent neural networks
12. Attention mechanisms
13. Test 2
14. Recoveries

Content

- Decision trees
- Voting
- Bagging and pasting
- Random forests
- Boosting

1) Decision trees



Lecture 10 - Decision Trees and Ensemble Methods | Stanford CS229: Machine Learning (Autumn 2018)

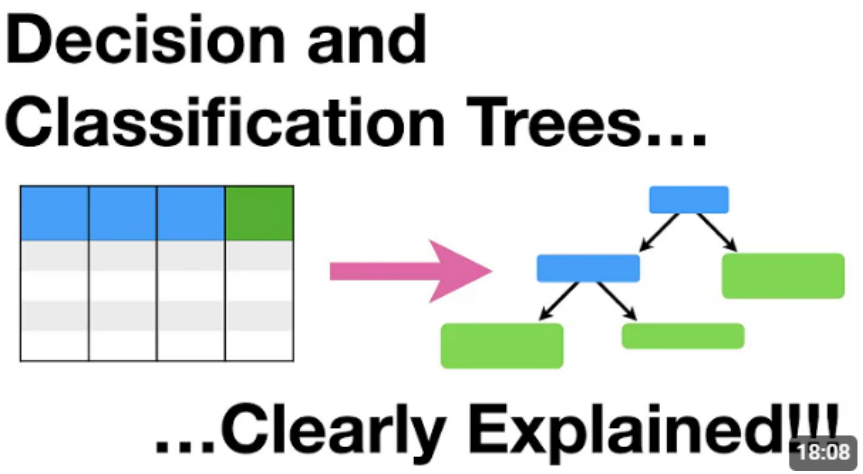
163 K de vizionări • acum 4 ani

Stanford Online

Raphael Townshend PhD Candidate and CS229 Head TA To follow along with the course schedule and syllabus, visit: ...

Subtitrări

Momente: 19 Decision Trees | Cross-Entropy Loss | The Cross Entropy Law | Miss Classification Loss | Gi



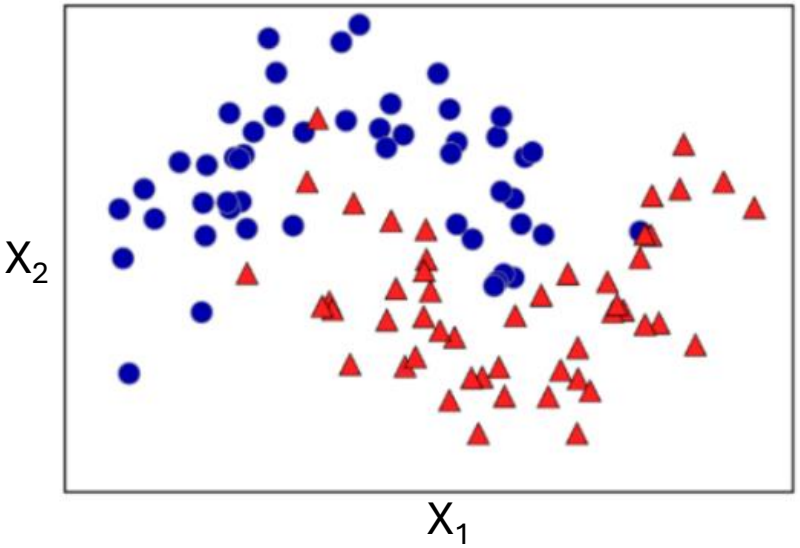
Decision and Classification Trees, Clearly Explained!!!

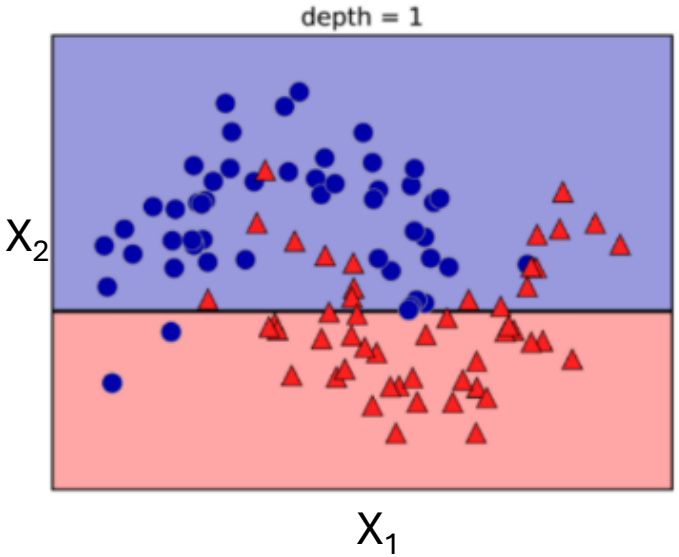
767 K de vizionări • acum 3 ani

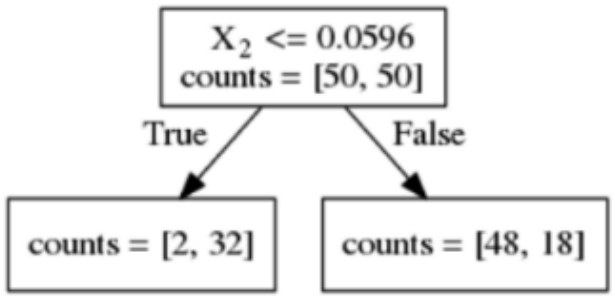
StatQuest with Josh Starmer

Decision trees are part of the foundation for Machine Learning. Although they are q

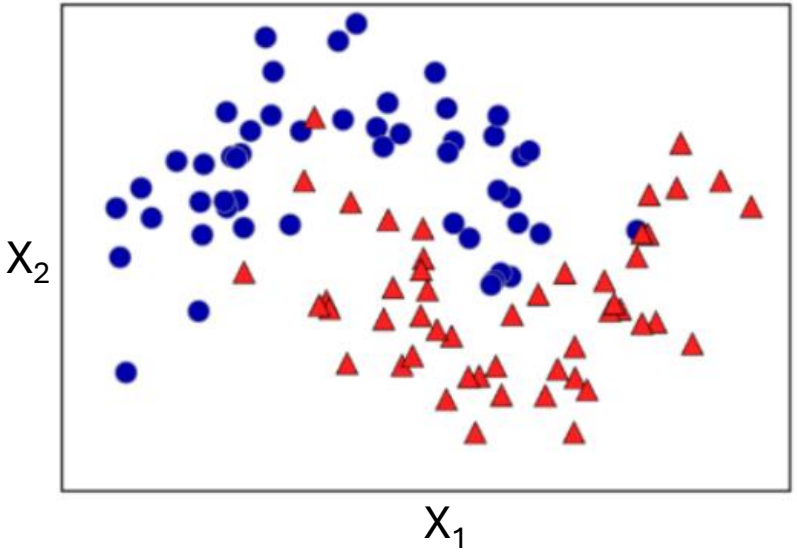
Capitole: 9 Awesome song and introduction | Basic decision tree con

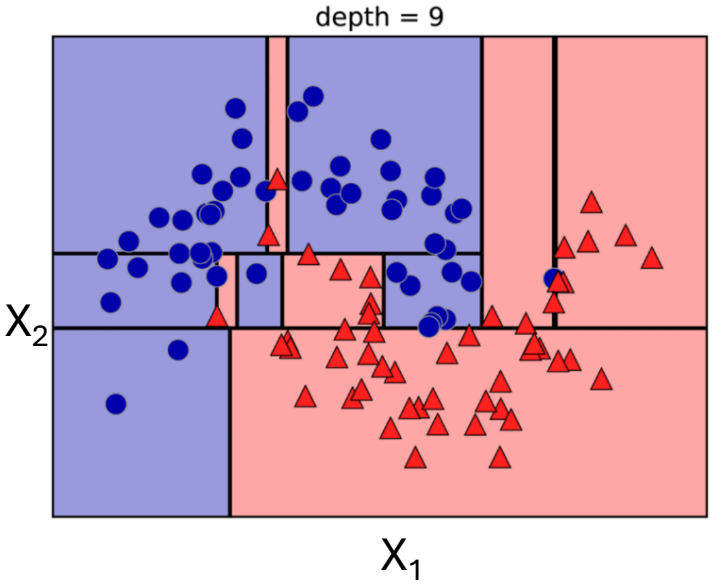


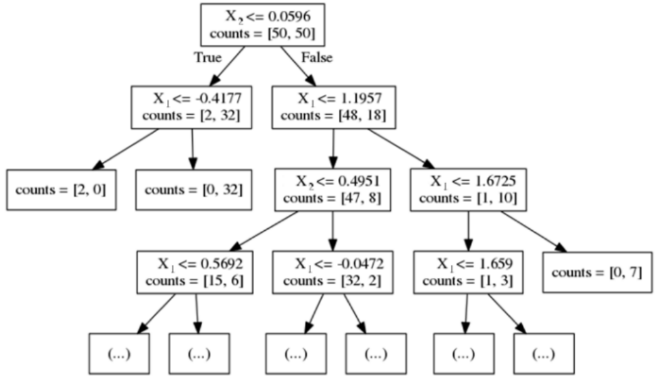




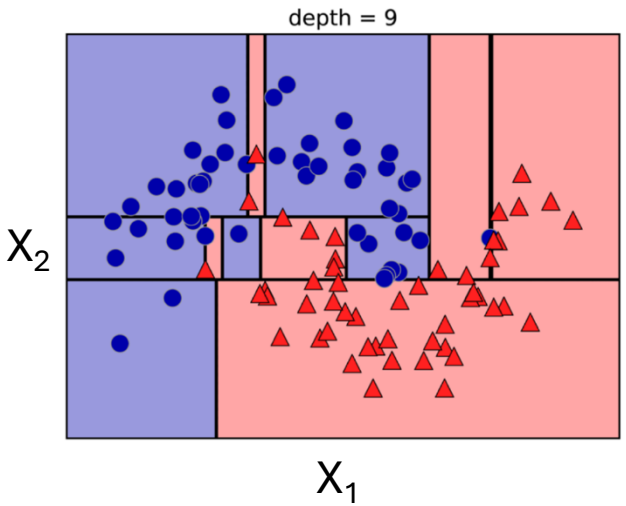
moons dataset

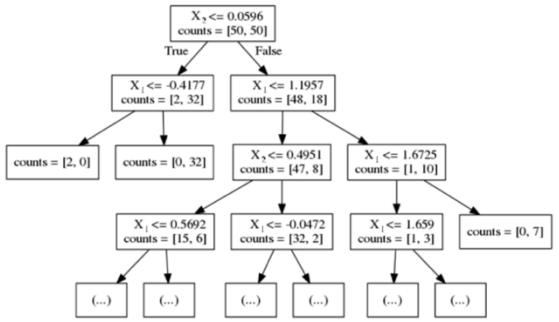






- learning a decision tree ➡ learning a sequence of *if/else questions*
- data partitioning is repeated until each leaf (terminal node) only contains a single target value

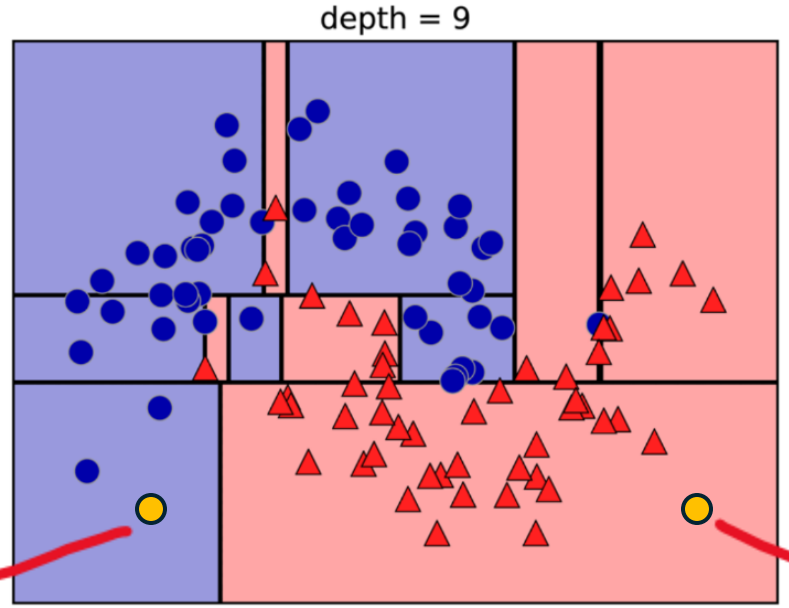


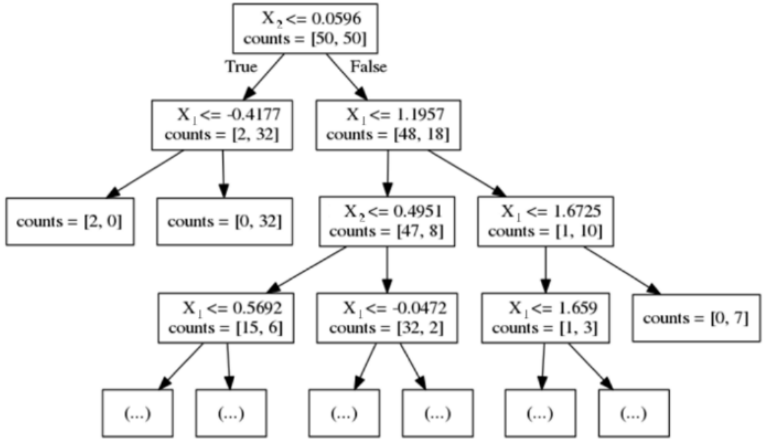


- a leaf of the tree that contains data points that all share the same target value is called *pure*

- a prediction on a new data point is made by checking which region of the partition of the feature space the point lies in and then:
  - predicting the majority target (or the single target in the case of pure leaves) in that region in the case of classification
  - predicting the mean target of the training points in the case of regression

Example:





if after traversing the tree from the root to the leaves the yellow test point is here, then:

- classification → the test point will become blue
- regression → the test point will be the mean of the 2 blue points

if after traversing the tree from the root to the leaves the yellow test point is here, then:

- classification → the test point will become red
- regression → the test point will be the mean of the 33 red points

Some real-world use case examples…

Heart Disease Prediction Using Classification with Different Decision Tree Techniques

K. Thenmozhi [1], P.Deepika[2]

[1]Asst. Professor, Department of Computer Science, Dr. N.G.P. Arts and Science College, CBE

[2]Asst. Professor, Department of Computer Science and Applications, Sasurie College of Arts and Science, CBE

Predicting Breast Cancer Survivability Using Fuzzy Decision Trees for Personalized Healthcare

Muhammad Umer Khan, Jong Pill Choi, Hyunjung Shin and Minkoo Kim

A Novel Idea for Credit Card Fraud Detection using Decision Tree

Prajal Save
St. John College of Engineering and Management
Palghar

Pranali Tiwarekar
St. John College of Engineering and Management
Palghar

Ketan N. Jain
St. John College of Engineering and Management
Palghar

Neha Mahyavanshi
St. John College of Engineering and Management
Palghar

A Decision Tree System for Finding Genes in DNA

Steven Salzberg* Arthur L. Delcher† Kenneth H. Fasman‡
John Henderson§

Code part

Classification

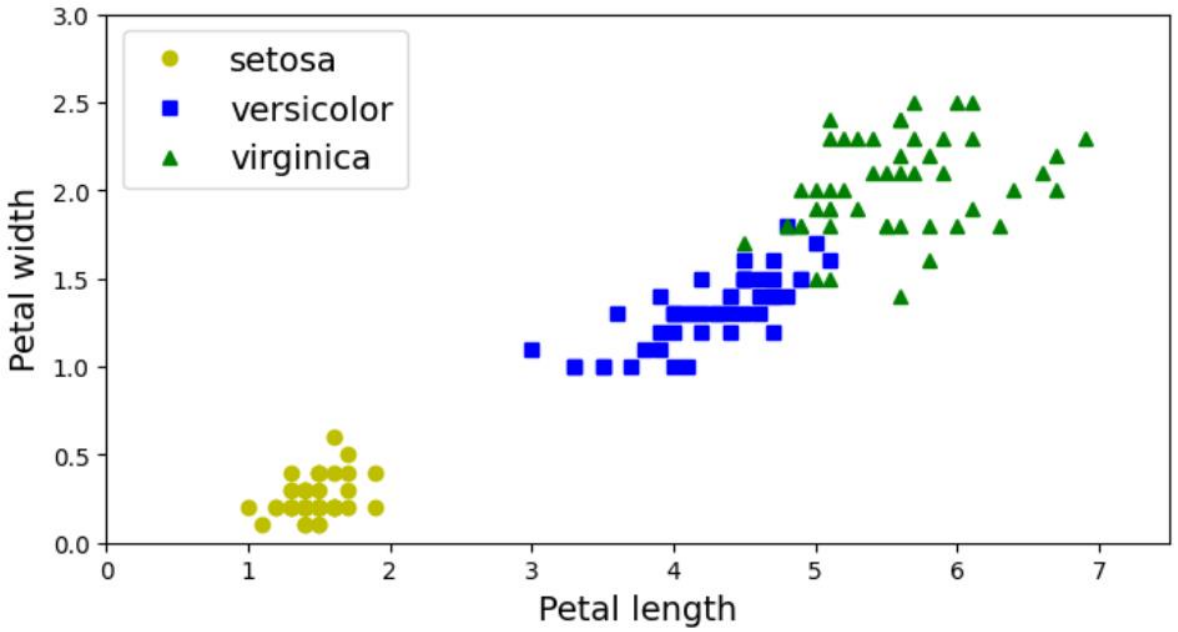
```python
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris_dataset = load_iris()
X = iris_dataset.data[:, 2:] # petal length and width
y = iris_dataset.target
```

```python
y
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

```python
X
```

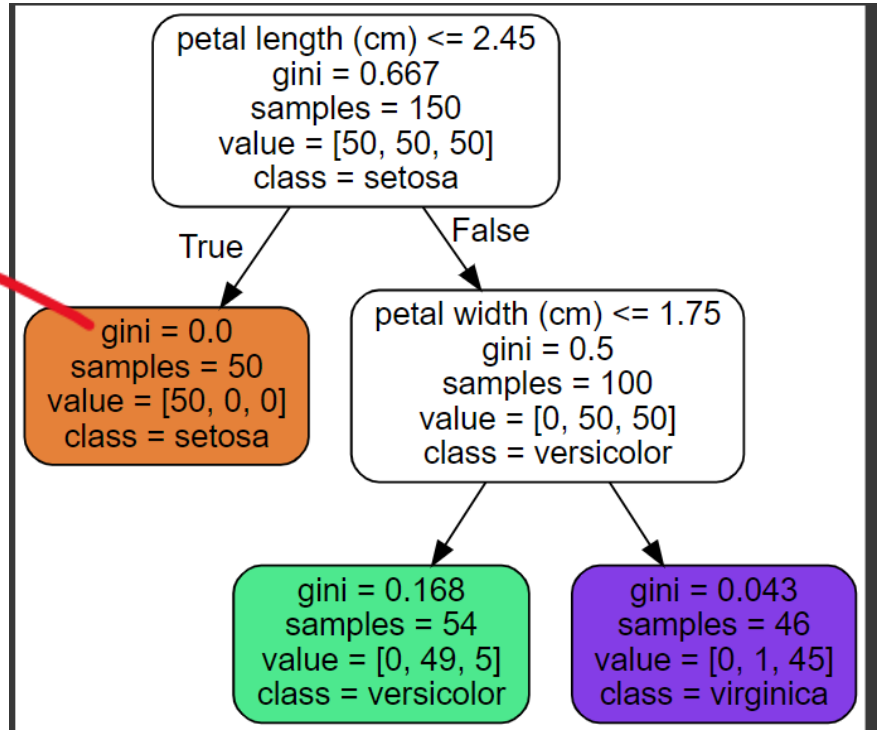
```
array([[1.4, 0.2],
       [1.4, 0.2],
       [1.3, 0.2],
       [1.5, 0.2],
       [1.4, 0.2],
       [1.7, 0.4],
       [1.4, 0.3],
       [1.5, 0.2],
       [1.4, 0.2],
```



```python
tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf.fit(X, y)
```

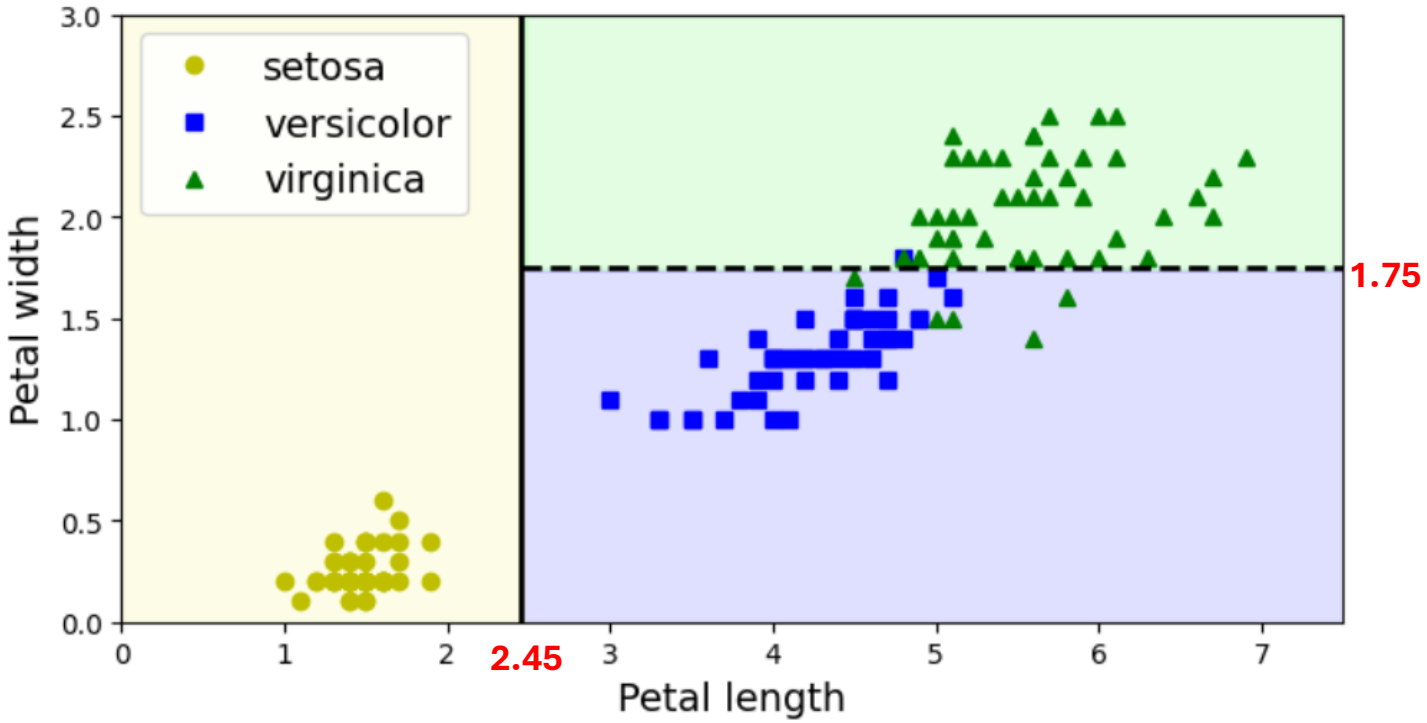
```python
export_graphviz(
        tree_clf,
        out_file=os.path.join(".", "iris_tree.dot"),
        feature_names=iris_dataset.feature_names[2:],
        class_names=iris_dataset.target_names,
        rounded=True,
        filled=True
    )

Source.from_file(os.path.join(".", "iris_tree.dot"))
```

gini attribute measure the impurity of the node: gini = 0 means a pure node



```
tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf.fit(X, y)
```

plot_decision_boundary() function already done in lab



Prediction:

```
tree_clf.predict_proba([[5, 1.5]])
```

```
array([[0.        , 0.90740741, 0.09259259]])
```

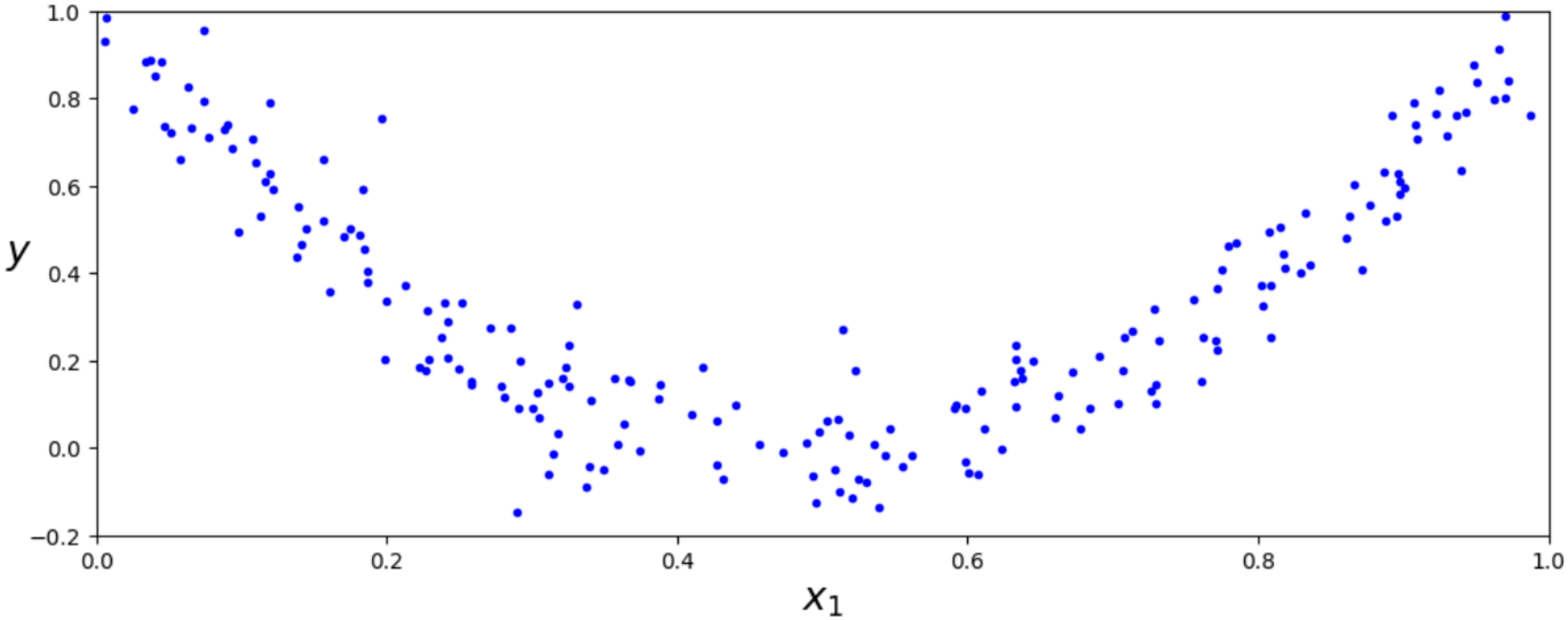
```
tree_clf.predict([[5, 1.5]])
```

```
array([1])
```

iris versicolor (class 1) because it has the highest probability.

Regression

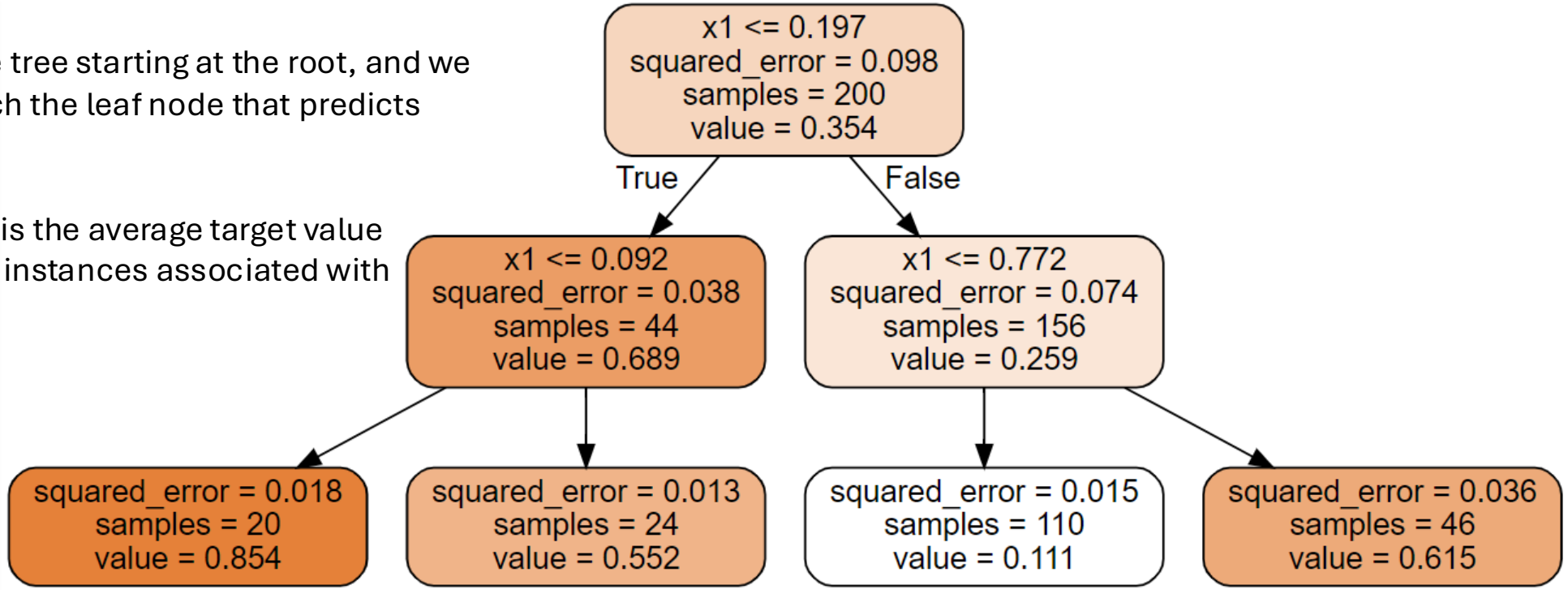
```python
# Quadratic training set + noise
np.random.seed(42)
n = 200
X = np.random.rand(n, 1)
y = 4 * (X - 0.5) ** 2
y = y + np.random.randn(n, 1) / 10
```

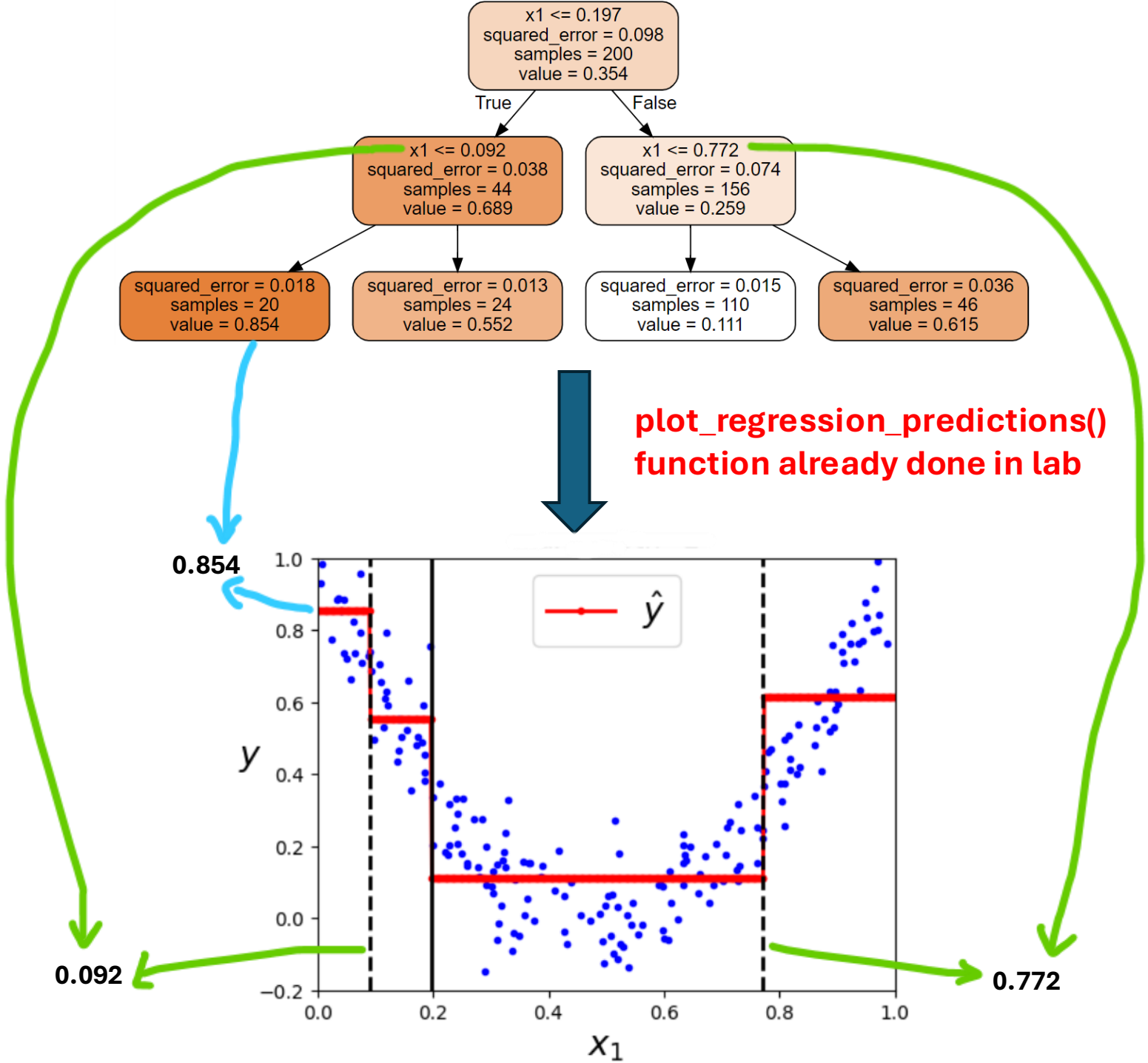


```python
from sklearn.tree import DecisionTreeRegressor

tree_reg = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg.fit(X, y)
```

- very similar to the classification tree we built earlier
- the main difference is that, instead of predicting a class in each node, it predicts a value
- for example, suppose we want to make a prediction for a new instance with x_1**=0.6**
- we traverse the tree starting at the root, and we eventually reach the leaf node that predicts **value=0.111**
- this prediction is the average target value of the 110 training instances associated with this leaf node

```python
export_graphviz(
        tree_reg,
        out_file=os.path.join(".", "regression_tree.dot"),
        feature_names=["x1"],
        rounded=True,
        filled=True
    )

Source.from_file(os.path.join(".", "regression_tree.dot"))
```




x1 <= 0.197
squared_error = 0.098
samples = 200
value = 0.354
True
False
x1 <= 0.092
squared_error = 0.038
samples = 44
value = 0.689
x1 <= 0.772
squared_error = 0.074
samples = 156
value = 0.259
squared_error = 0.018
samples = 20
value = 0.854
squared_error = 0.013
samples = 24
value = 0.552
squared_error = 0.015
samples = 110
value = 0.111
squared_error = 0.036
samples = 46
value = 0.615
plot_regression_predictions()
function already done in lab
0.854
0.092
0.772
$\hat{y}$
y
x_1
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
0.0
0.2
0.4
0.6
0.8
1.0

2) Voting

- if we aggregate the predictions of a group of predictors (classifiers or regressors), we will often get better predictions than with the best individual predictor

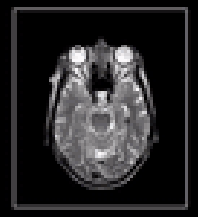
- a group of predictors is called **ensemble**

- suppose we have trained a few classifiers, each one achieving about 80% accuracy

- a simple way to create an even better classifier (maybe 85% accuracy) is to aggregate the predictions of each classifier and predict the class that gets the most votes

Ensemble Learning Intuition

Real Life Example



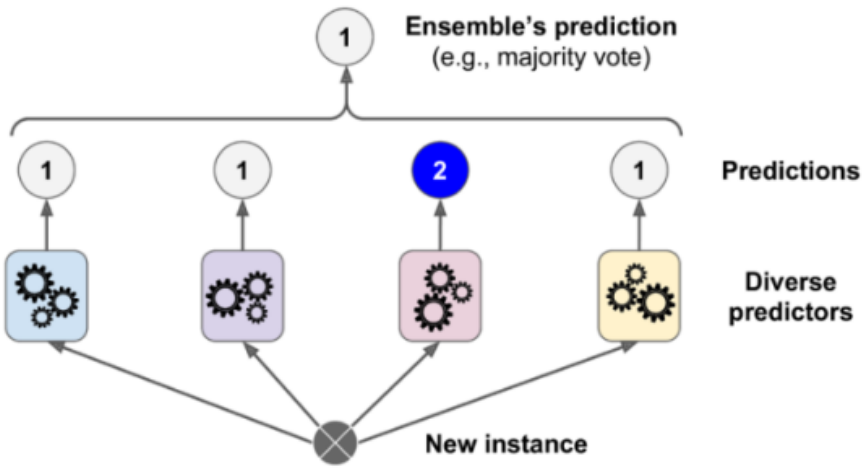
MRI Scan

Doc 1

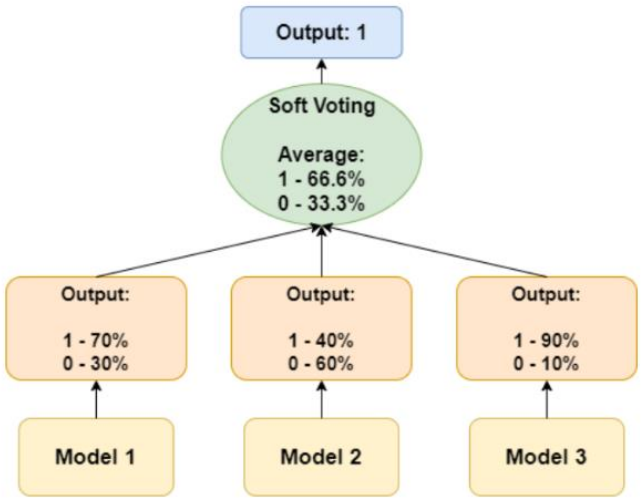
Doc 2

Doc 3

hard voting classifier → aggregate the predictions of each classifier and predict the class that gets the most votes:



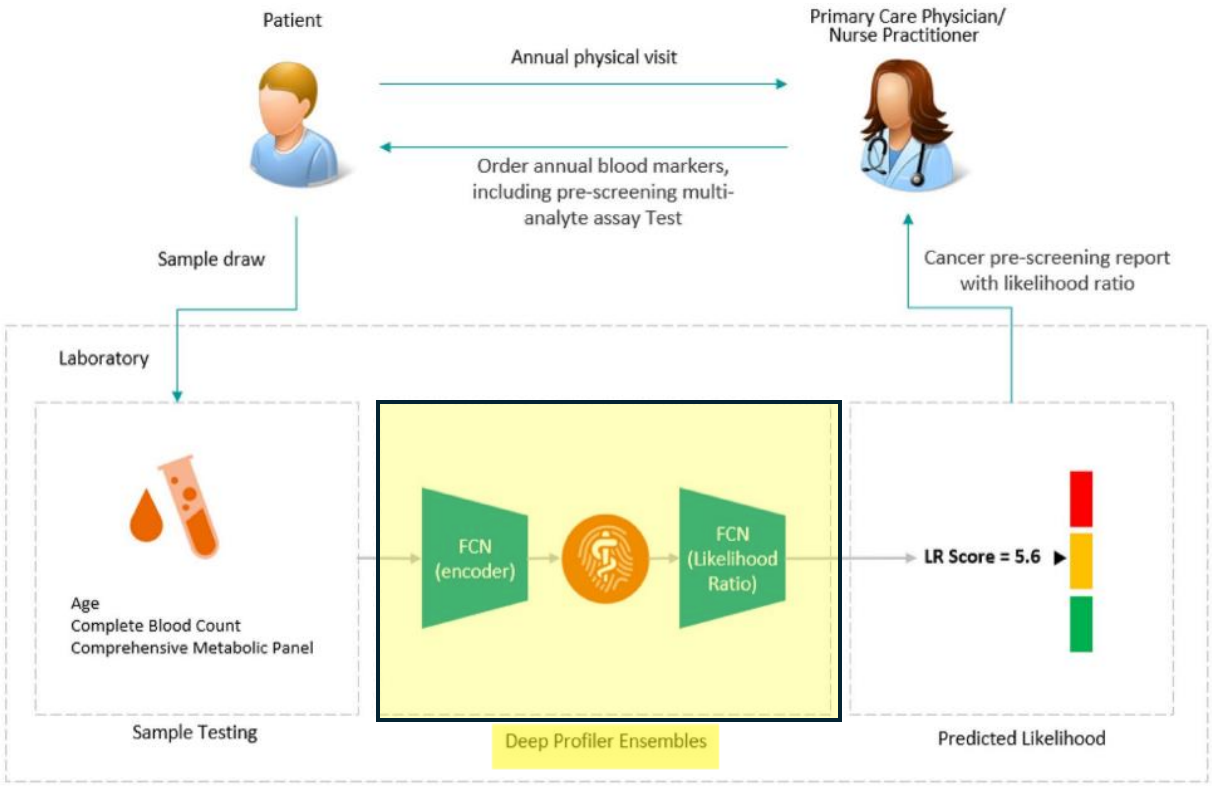
soft voting classifier → if all classifiers are able to estimate class probabilities and we predict the class with the highest class probability averaged over all the individual classifiers:



Example from Previous Work: Ensemble Model for Cancer Risk Prediction

Deep learning-based identification of patients at increased risk of cancer using routine laboratory markers

Vivek Singh[1]✉, Shikha Chaganti[1], Matthias Siebert[2], Sowmya Rajesh[1], Andrei Puiu[3,4], Raj Gopalan[5], Jamie Gramz[6], Dorin Comaniciu[1] & Ali Kamen[1]

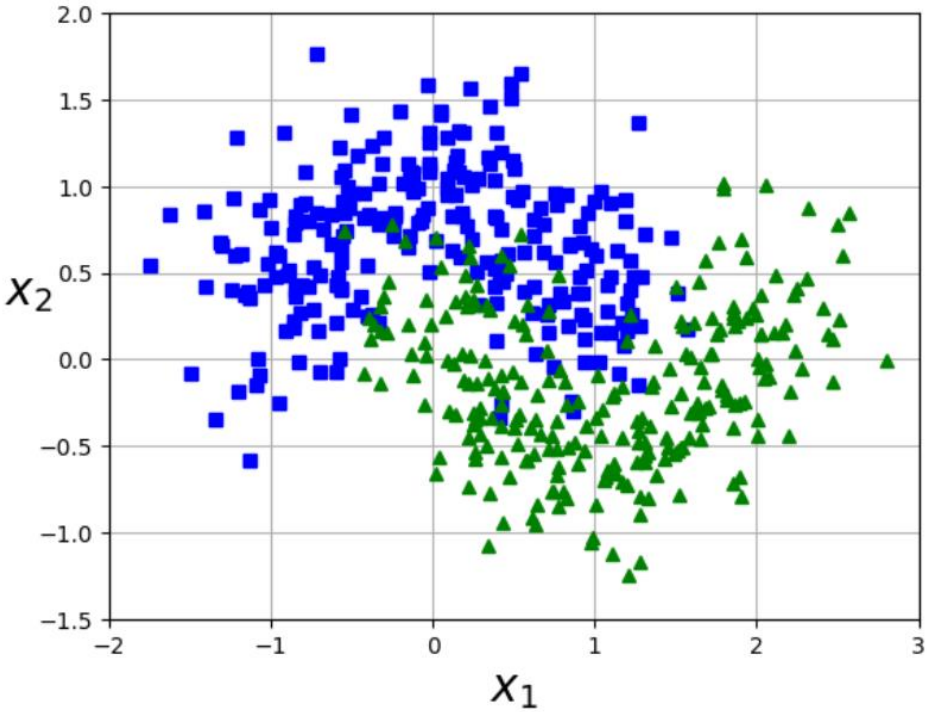


- **Patient gives a blood sample** → sent to the lab.
- **AI model (ensemble of neural networks)** analyzes the results.
- Each model (called a *Deep Profiler*) learns different medical patterns.
- Their outputs are **combined into one "likelihood ratio" score**.
- The score helps doctors decide if further testing is needed.

Code part

```python
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_moons

X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
```

plot_dataset() function from lab 3 ☺



Hard voting

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

log_clf = LogisticRegression(solver="lbfgs", random_state=42)
rnd_clf = RandomForestClassifier(n_estimators=100, random_state=42)
svm_clf = SVC(gamma="scale", random_state=42)

voting_clf = VotingClassifier(
    estimators=[('lr', log_clf), ('rf', rnd_clf), ('svc', svm_clf)],
    voting='hard')
```

```python
voting_clf.fit(X_train, y_train)
```

```python
from sklearn.metrics import accuracy_score

for clf in (log_clf, rnd_clf, svm_clf, voting_clf):
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    print(clf.__class__.__name__, accuracy_score(y_test, y_pred))
```

```
LogisticRegression 0.864
RandomForestClassifier 0.896
SVC 0.896
VotingClassifier 0.912
```

Soft voting

```
log_clf = LogisticRegression(solver="lbfgs", random_state=42)
rnd_clf = RandomForestClassifier(n_estimators=100, random_state=42)
svm_clf = SVC(gamma="scale", probability=True, random_state=42)

voting_clf = VotingClassifier(
    estimators=[('lr', log_clf), ('rf', rnd_clf), ('svc', svm_clf)],
    voting='soft')
voting_clf.fit(X_train, y_train)
```

```
from sklearn.metrics import accuracy_score

for clf in (log_clf, rnd_clf, svm_clf, voting_clf):
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    print(clf.__class__.__name__, accuracy_score(y_test, y_pred))
```

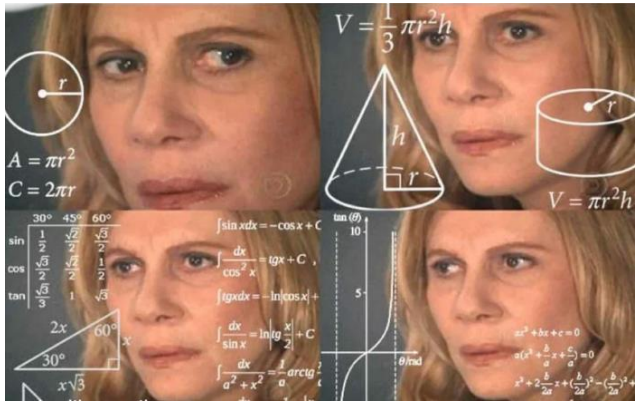
```
LogisticRegression 0.864
RandomForestClassifier 0.896
SVC 0.896
VotingClassifier 0.92
```

3) Bagging and pasting

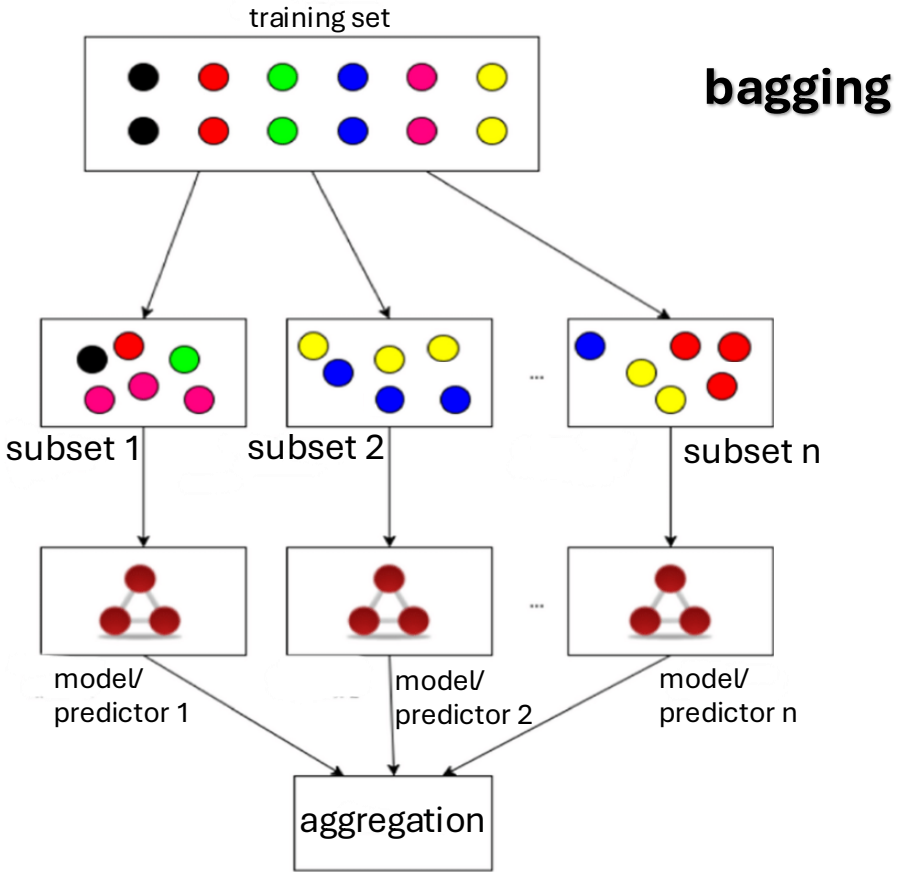
- use the same training algorithm for every model/predictor but train them on different random subsets of the training set

often called **bootstrap**

- when sampling is performed **with replacement** → **bagging**
- when sampling is performed **without replacement** → **pasting**

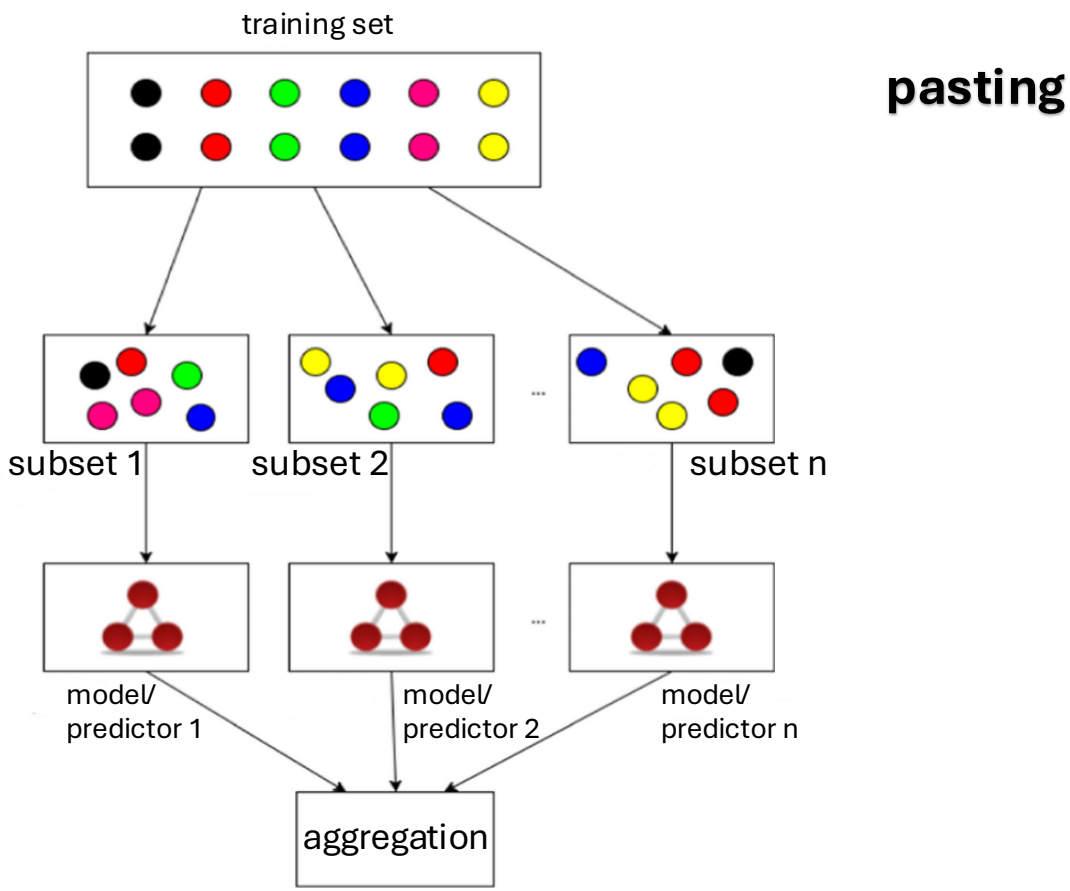


- once all models/predictors are trained, the ensemble can make a prediction for a new instance by simply aggregating the predictions of all models/predictors



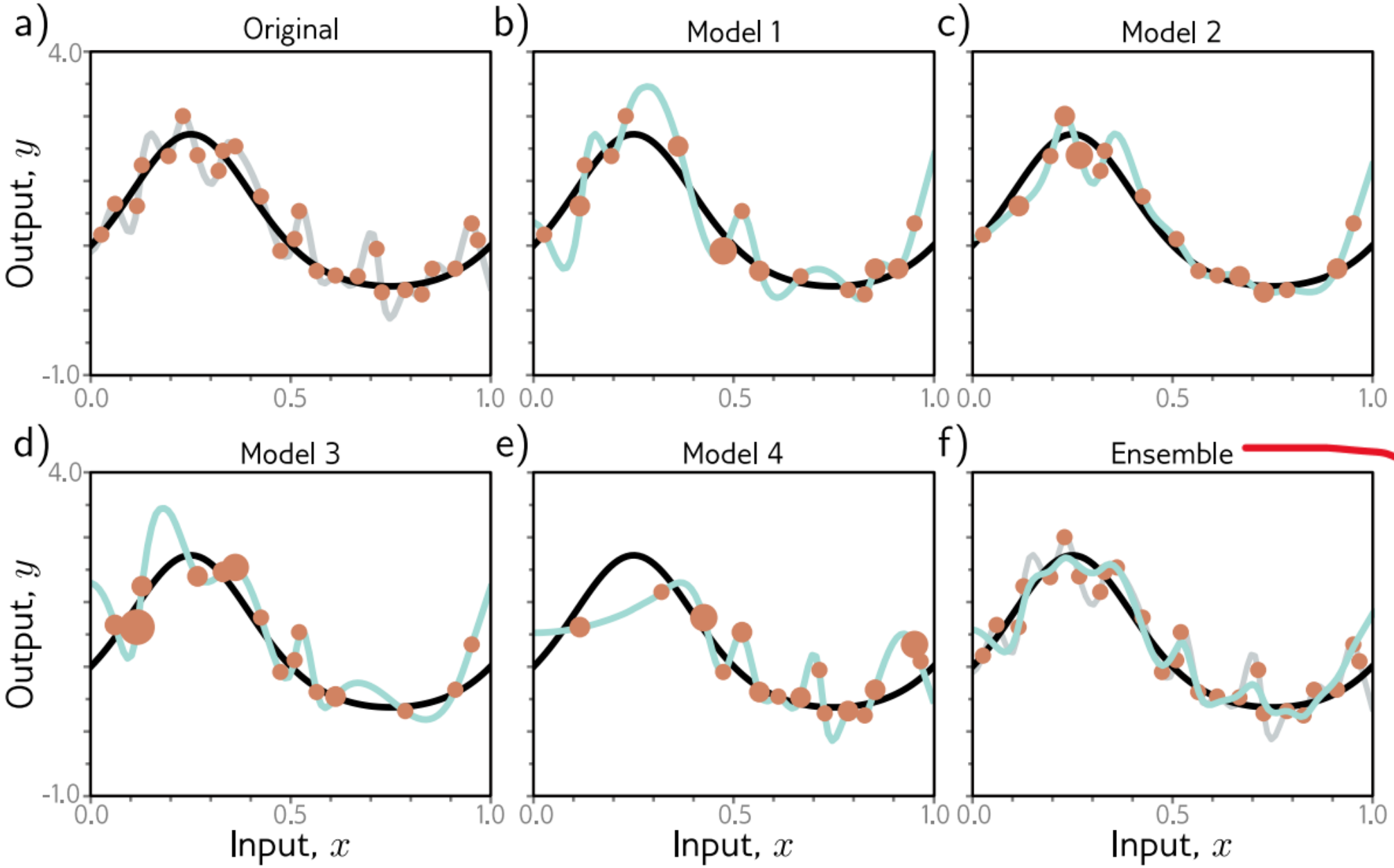
- suppose you have a dataset with 12 points
- for each model, you want to draw **6 random samples** from these 12 points.
- if you're **sampling with replacement**, then for each model:

  1. randomly select a data point from the 12 data points.
  2. record that selection, but then **put that data point back into the pool** of 12 data points, **so it can be selected again**.
  3. repeat this process 6 times.



- suppose you have a dataset with 12 points
- for each model, you want to draw **6 random samples** from these 12 points.
- if you're **sampling without replacement**, then for each model:

  1. randomly select a data point from the 12 data points.
  2. record that selection, but **do not put that data point back into the pool** of 12 data points, **so it can't be selected again**.
  3. repeat this process 6 times.



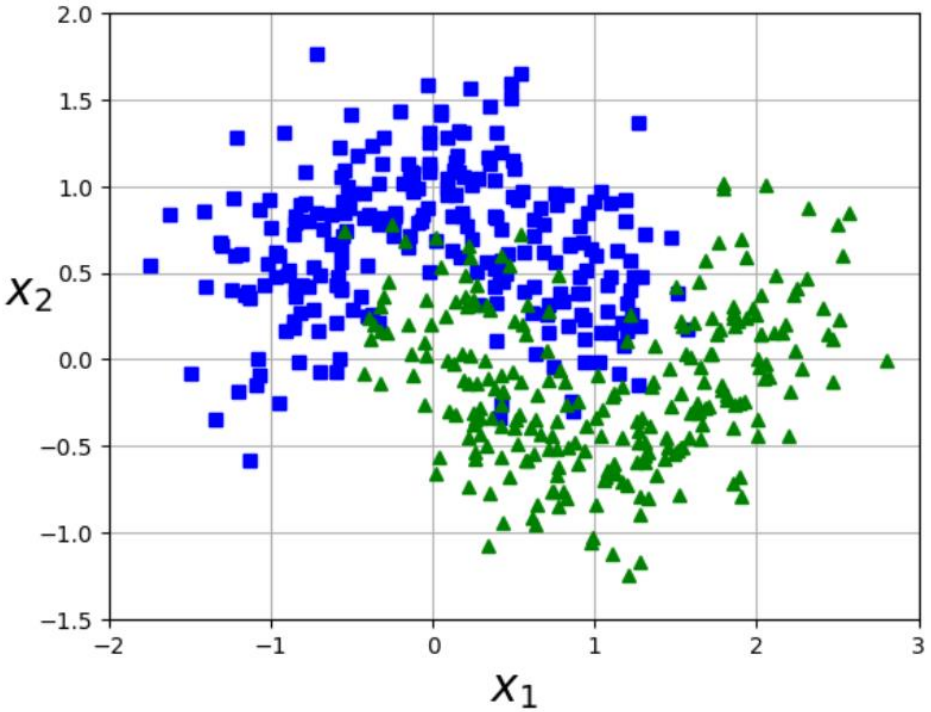
No more wiggle wiggle ☺

Averaging multiple “wiggly” models can reduce **overfitting** and variance, leading to **smoother** and more **stable** predictions!

Code part

```python
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_moons

X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
```

plot_dataset() function from lab 3 ☺



Bagging

```python
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bag_clf = BaggingClassifier(
    DecisionTreeClassifier(), n_estimators=500,
    max_samples=100, bootstrap=True, random_state=42)
bag_clf.fit(X_train, y_train)
y_pred = bag_clf.predict(X_test)
```

each decision tree is trained on 100 training instances

use an ensemble of 500 decision tree

the 100 training instances are randomly sampled from the training set **with replacement** → **bagging**

accuracy for a single decision tree classifier:

```python
tree_clf = DecisionTreeClassifier(random_state=42)
tree_clf.fit(X_train, y_train)
y_pred_tree = tree_clf.predict(X_test)
print(accuracy_score(y_test, y_pred_tree))
```

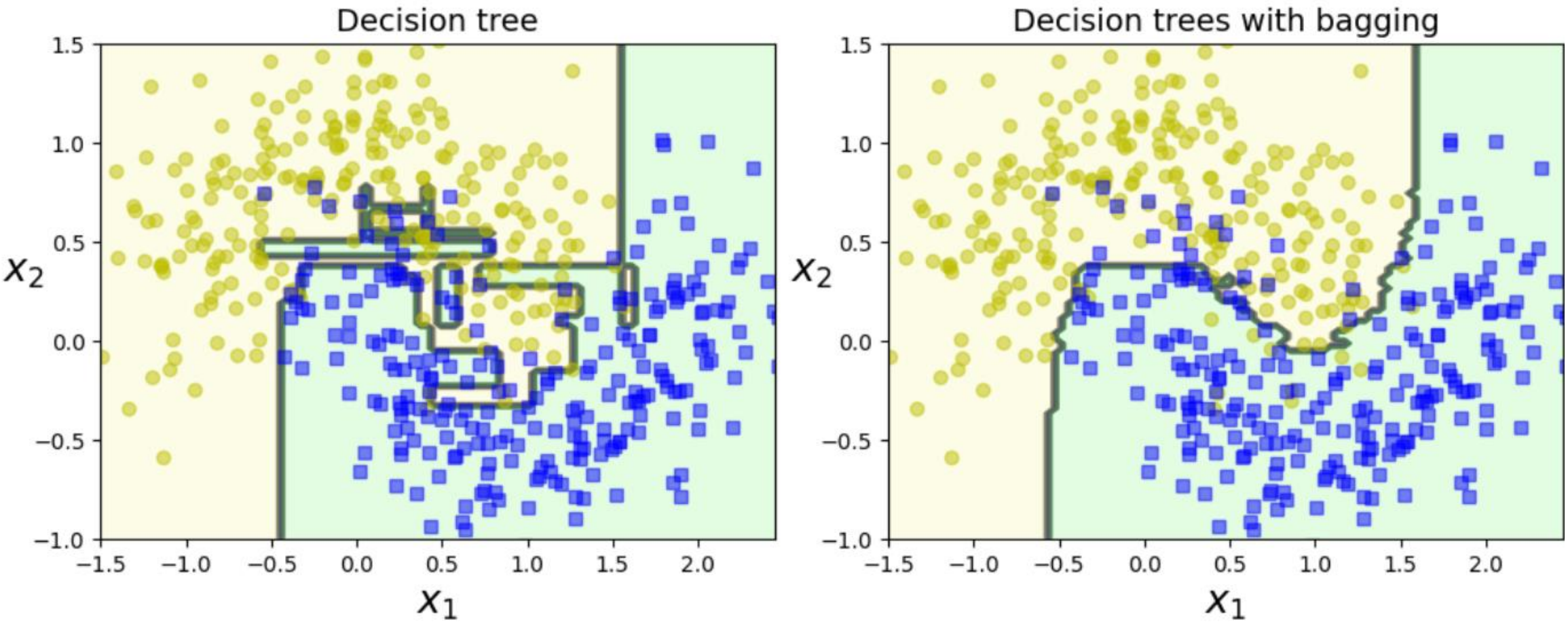
```
0.856
```

accuracy the bagging classifier:

```python
from sklearn.metrics import accuracy_score
print(accuracy_score(y_test, y_pred))
```

```
0.904
```

plot_decision_boundary()
function already done in lab



Pasting

```python
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bag_clf = BaggingClassifier(
    DecisionTreeClassifier(), n_estimators=500,
    max_samples=100, bootstrap=False, random_state=42)
bag_clf.fit(X_train, y_train)
y_pred = bag_clf.predict(X_test)
```

each decision tree is trained on 100 training instances

use an ensemble of 500 decision tree

the 100 training instances are randomly sampled from the training set **without replacement** → **pasting**

accuracy for a single decision tree classifier:

```python
tree_clf = DecisionTreeClassifier(random_state=42)
tree_clf.fit(X_train, y_train)
y_pred_tree = tree_clf.predict(X_test)
print(accuracy_score(y_test, y_pred_tree))
```

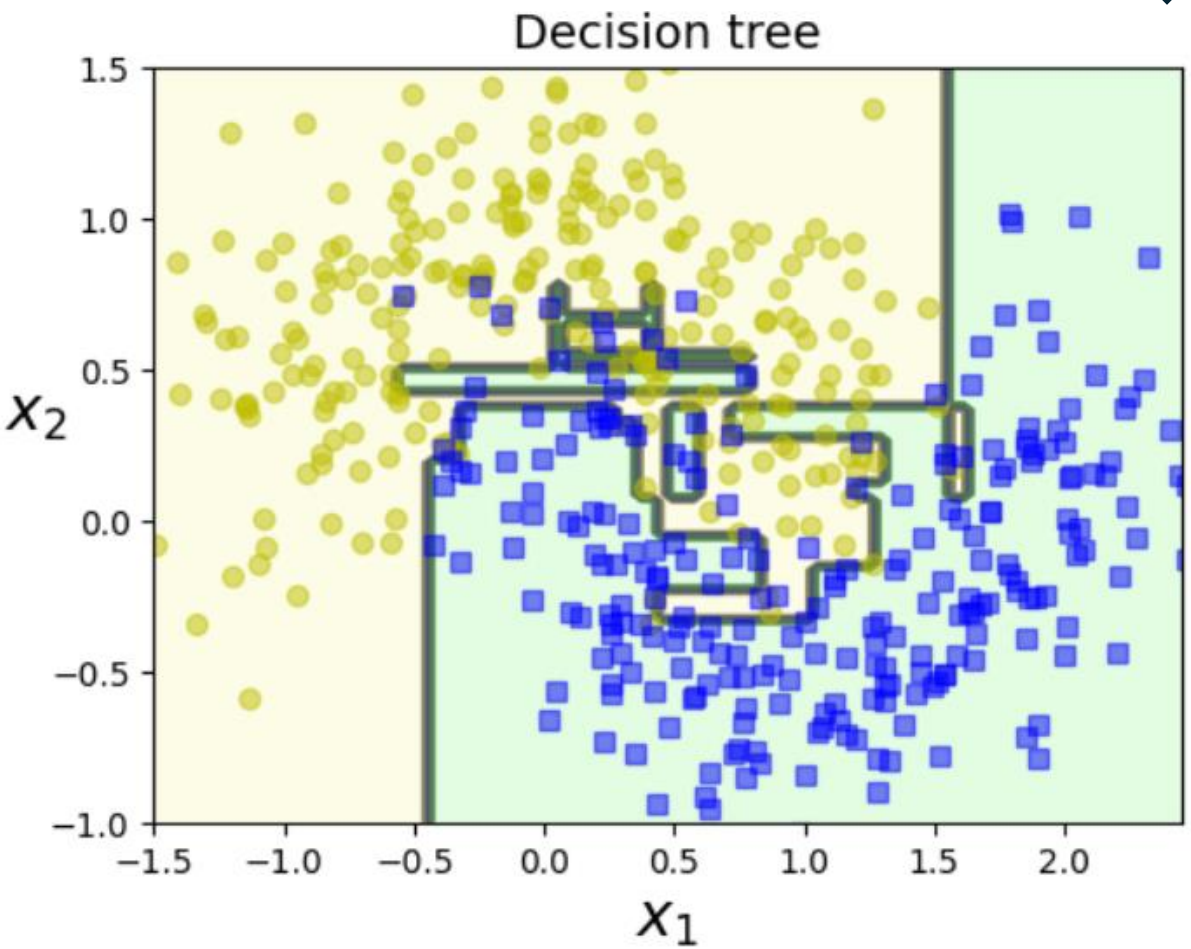
```
0.856
```

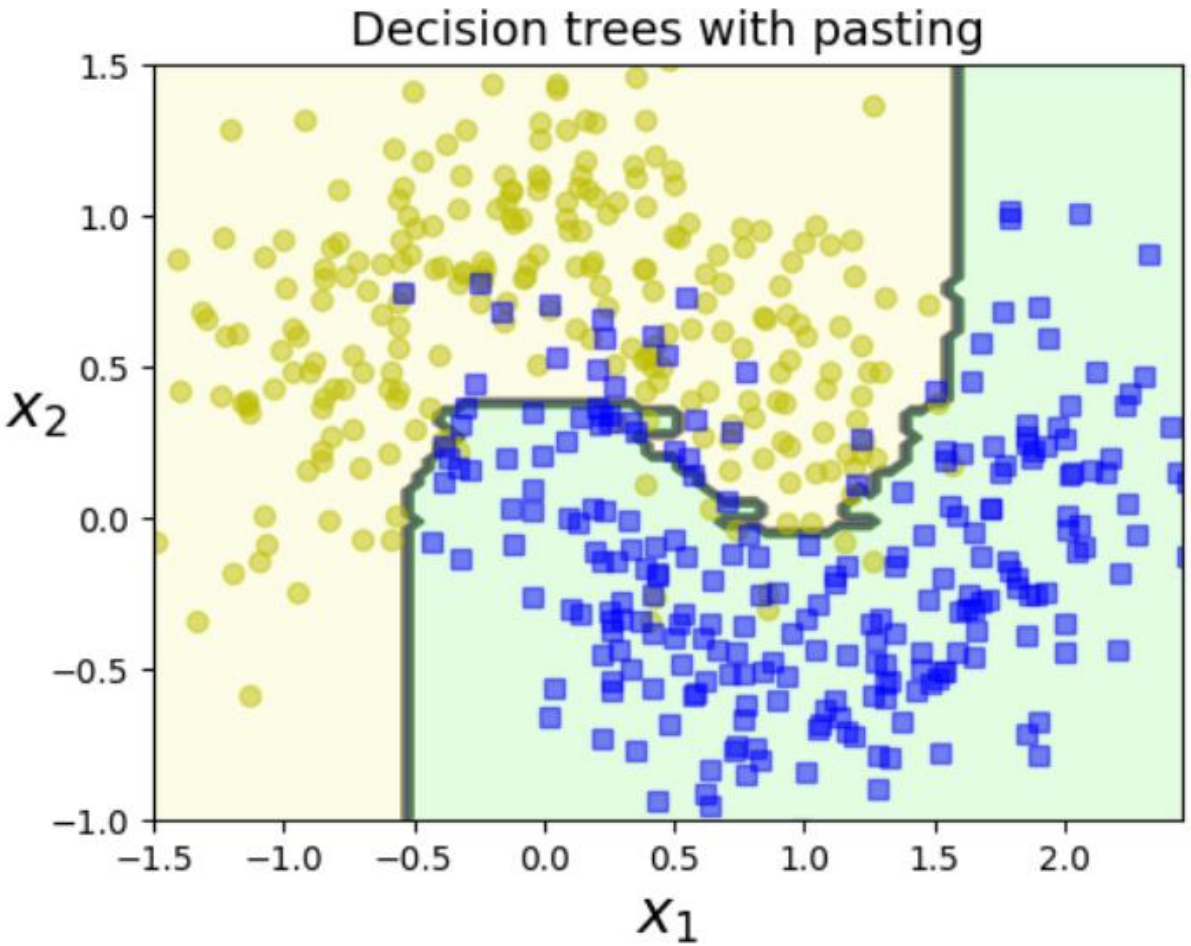
accuracy the pasting classifier:

```python
from sklearn.metrics import accuracy_score
print(accuracy_score(y_test, y_pred))
```

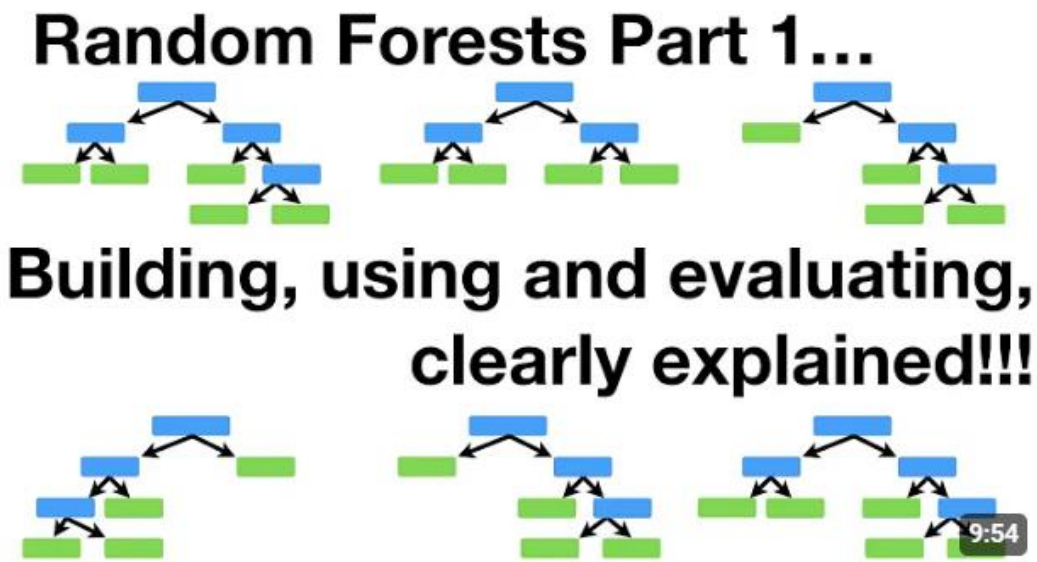
```
0.92
```

plot_decision_boundary()
function already done in lab





4) Random forests



StatQuest: Random Forests Part 1 - Building, Using and Evaluating

1,1 mil. de vizionări • acum 6 ani

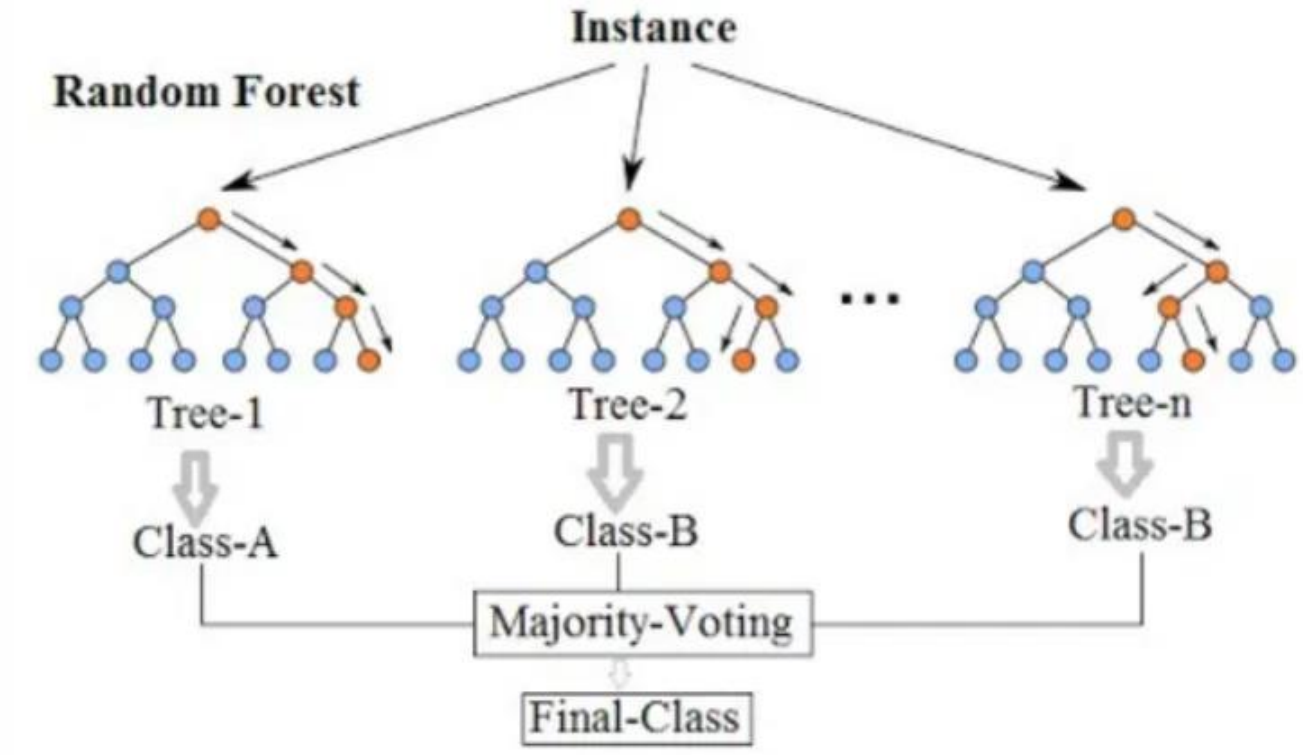
StatQuest with Josh Starmer

Random Forests make a simple, yet effective, machine learning method. They are made out of dec

Subtitrări

Capitole: 9 Awesome song and introduction | Motivation for using Random Forests |

- an ensemble of decision trees, generally trained via the bagging method (or sometimes pasting), typically with maximum number of samples set to the size of the training set

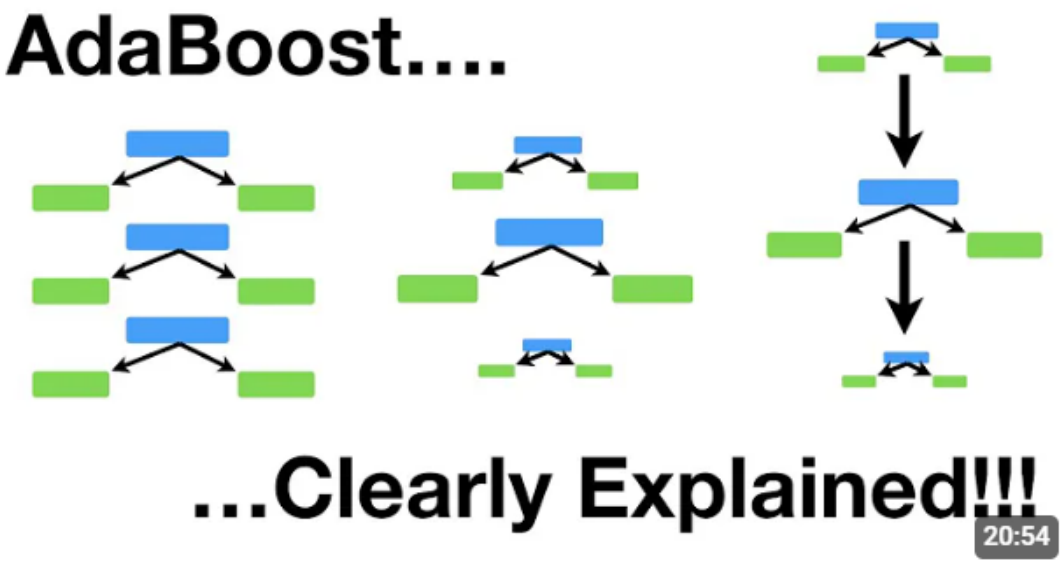


Code part

don’t worry about them ☺

```
from sklearn.ensemble import RandomForestClassifier

rnd_clf = RandomForestClassifier(n_estimators=500, max_leaf_nodes=16, random_state=42
rnd_clf.fit(X_train, y_train)

y_pred_rf = rnd_clf.predict(X_test)
```

5) Boosting



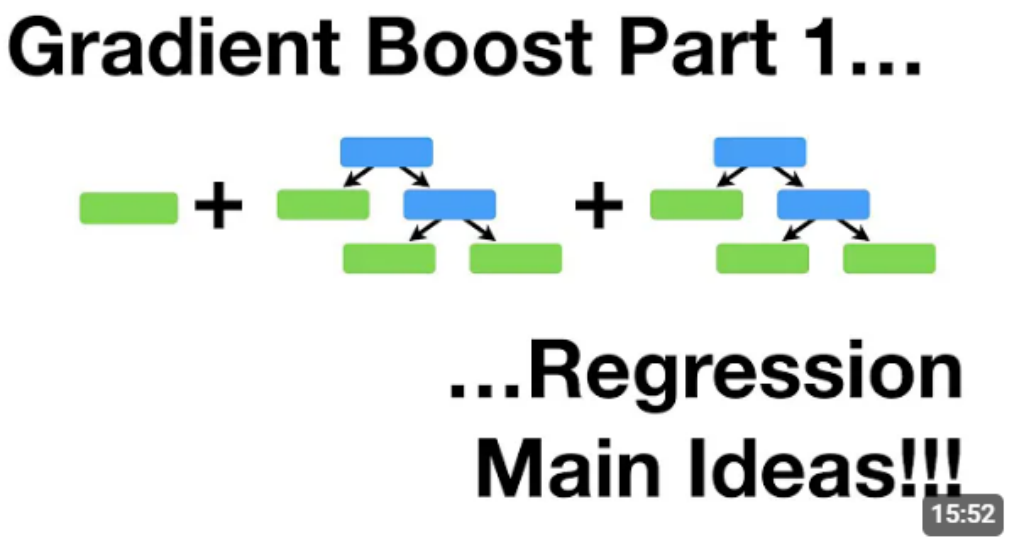
AdaBoost, Clearly Explained

767 K de vizionări • acum 5 ani

StatQuest with Josh Starmer

AdaBoost is one of those machine learning methods that seems s

Capitole: 10 Awesome song and introduction | The



Gradient Boost Part 1 (of 4): Regression Main Ideas

826 K de vizionări • acum 5 ani

StatQuest with Josh Starmer

Gradient Boost is one of the most popular Machine Learning algorithms in use. A

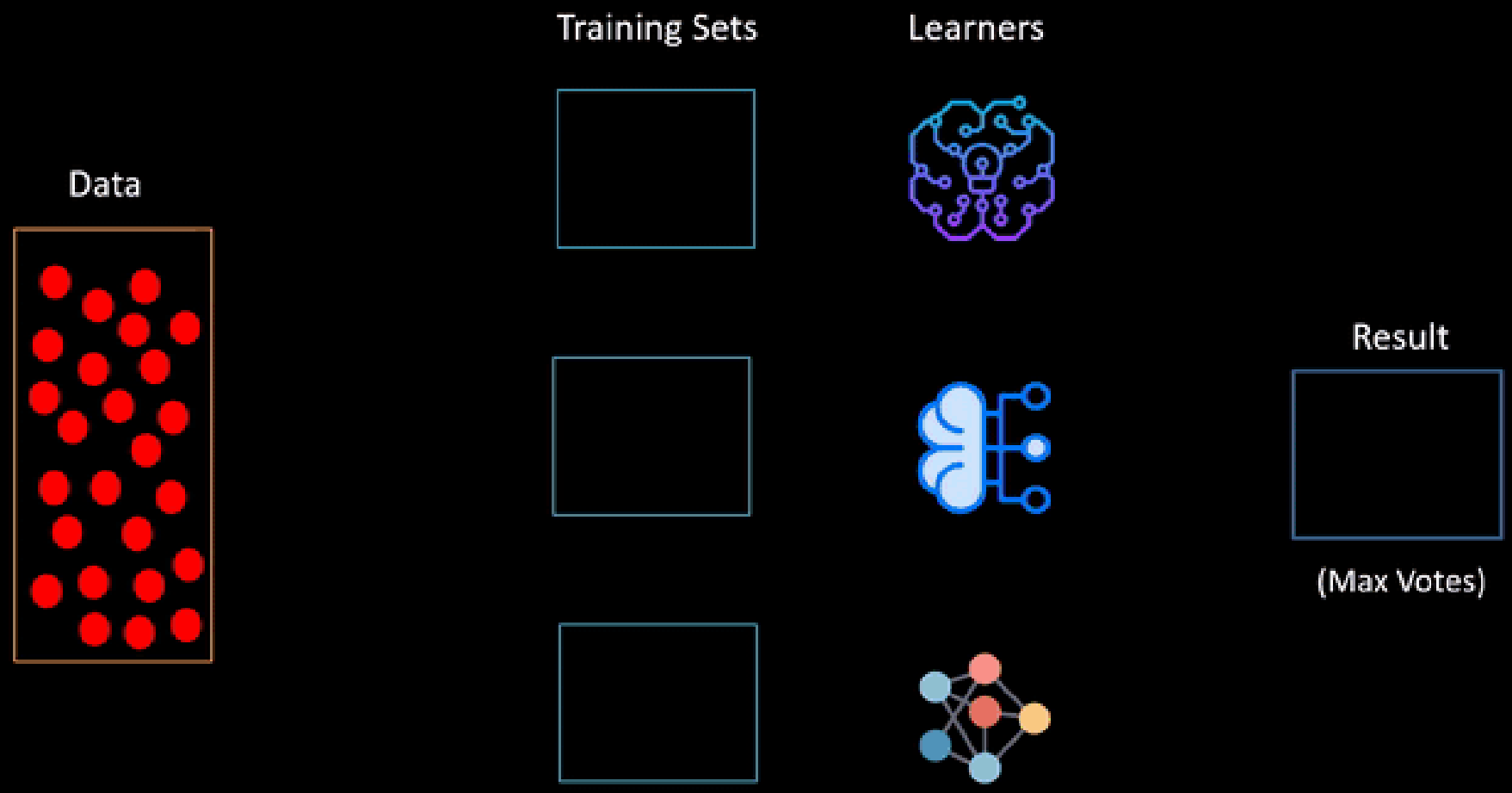
Capitole: 7 Awesome song and introduction | Gradient Boost comp

- general idea of **boosting** methods: train models/predictors sequentially, each trying to correct its predecessor

- **AdaBoost** → new models/predictors pay more attention to the training instances that the predecessor underfitted

- **gradient boosting** → new models/predictors try to fit to the residual errors made by the previous predictor

Boosting - Intuition

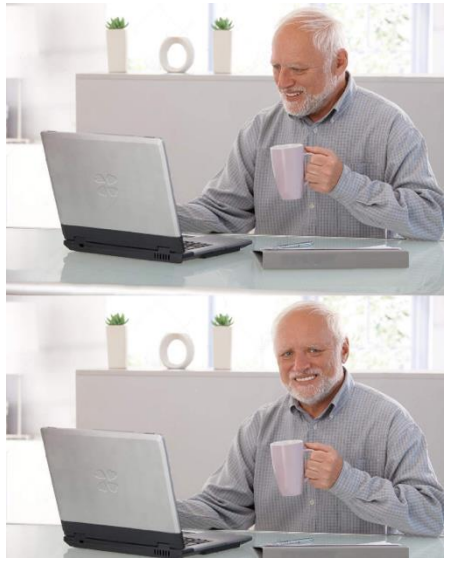


AdaBoost steps:

- first train a base classifier (such as a decision tree) and use it to make predictions on the training set

- increase the relative weights of misclassified training instances

- train a second classifier using the updated weights, use it to make predictions on the training set, update the instance weights and so on



Assume we have the following toy dataset:

Sample	Feature 1	Feature 2	Label
1	2	2	0
2	1	2	0
3	2	3	1
4	3	3	1

Step 1: Initialize Sample Weights

Initially, all samples have equal weights because we don't know which are harder to classify.

1)

Sample	Weight (w)
1	0.25
2	0.25
3	0.25
4	0.25

The weights for all samples start at **1/N** (where **N** is the total number of samples, 4 in this case).

Step 2: Train the First Weak Classifier

We now train a weak classifier (e.g., a decision stump) on the weighted data. For simplicity, let's assume that our first weak classifier looks at **Feature 1** and classifies samples as:

- If Feature 1 ≤ 1.5 → Class 0
- Otherwise → Class 1

2)

Using this rule, the weak classifier makes the following predictions:

Sample	True Label	Prediction (Weak Classifier 1)
1	0	1
2	0	0
3	1	1
4	1	1

- Sample 1 is misclassified (True Label = 0, Prediction = 1).
- All other samples (Samples 2, 3, and 4) are classified correctly.

Step 3: Calculate Error Rate (Error)

The error rate is calculated as the sum of the weights of the misclassified samples. In this case:

- Sample 1 is misclassified, so the error is:

$$\text{Error} = w_1 = 0.25$$

3, 4)

Step 4: Compute Classifier Weight (α)

The weight of the classifier α reflects its performance. It is calculated using the following formula:

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \text{Error}}{\text{Error}} \right)$$

Substituting the error value:

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - 0.25}{0.25} \right) = \frac{1}{2} \ln (3) \approx 0.55$$

Step 5: Update Weights

Now, we update the weights of the samples. Misclassified samples will have their weights increased, and correctly classified samples will have their weights decreased. The new weight is calculated as:

$$w_{\text{new}} = w_{\text{old}} \times e^{\pm\alpha}$$

5)

- For **correctly classified** samples:

$$w_{\text{new}} = w_{\text{old}} \times e^{-\alpha}$$

- For **misclassified** samples:

$$w_{\text{new}} = w_{\text{old}} \times e^{+\alpha}$$

Sample	Weight (new, normalized)
1	0.433 / 0.865 ≈ 0.50
2	0.144 / 0.865 ≈ 0.17
3	0.144 / 0.865 ≈ 0.17
4	0.144 / 0.865 ≈ 0.17

6, 7)

Step 6: Train the Second Weak Classifier

Now we train a second weak classifier based on **Feature 2** with the rule:

- **If Feature 2 ≤ 2.5 → Class 0, Otherwise → Class 1**

Given the **Feature 2** values:

Sample	Feature 2	True Label	Prediction (Weak Classifier 2)
1	2	0	0 (correct)
2	2	0	0 (correct)
3	3	1	1 (correct)
4	3	1	1 (correct)

This classifier **classifies all samples correctly**.

Step 7: Calculate the Error Rate for Classifier 2

Since the second classifier classifies everything perfectly, the error rate is:

$$\text{Error} = 0$$

8, 9)

Step 8: Compute the Classifier Weight (α) for Classifier 2

Since the error is 0, the classifier weight α is very large, making this classifier highly influential in the final prediction. In theory, $\alpha \to \infty$, but in practice, we give it a large positive value to reflect its perfect classification performance.

Step 9: Final Prediction (Weighted Majority Vote)

Now, we combine the two classifiers' predictions using their weights:

- **Classifier 1 (α = +0.55):**
  - Sample 1: Predicted **1**
  - Sample 2: Predicted **0**
  - Sample 3: Predicted **1**
  - Sample 4: Predicted **1**
- **Classifier 2 (very large α):**
  - Sample 1: Predicted **0**
  - Sample 2: Predicted **0**
  - Sample 3: Predicted **1**
  - Sample 4: Predicted **1**

For each sample, we take the **weighted majority vote**:

Sample	Final Prediction
1	0 (dominated by Classifier 2)
2	0 (dominated by Classifier 2)
3	1 (both classifiers agree)
4	1 (both classifiers agree)

- **Scopul:**
  Să îmbunătățim un clasificator simplu (ex: un arbore mic de decizie) făcându-l să învețe mai mult din greșeli la fiecare pas.

- **Ideea de bază:**
  *În loc să dăm aceeași importanță tuturor exemplelor, AdaBoost **mărește ponderea** celor clasificate greșit — ca următorul model să fie mai atent la ele.*

- **Pașii principali:**

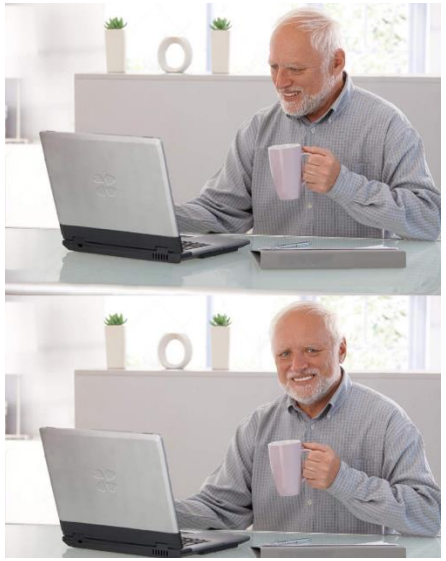
🔹 Antrenăm primul clasificator pe toate exemplele (cu ponderi egale).
🔹 Vedem ce exemple a greșit și **le creștem ponderea**.
🔹 Antrenăm un al doilea clasificator — acesta acordă **mai multă atenție** exemplelor dificile.
🔹 Repetăm procesul (ajustăm ponderi, adăugăm noi modele).
🔹 Predicția finală = combinația ponderată a tuturor clasificatorilor.

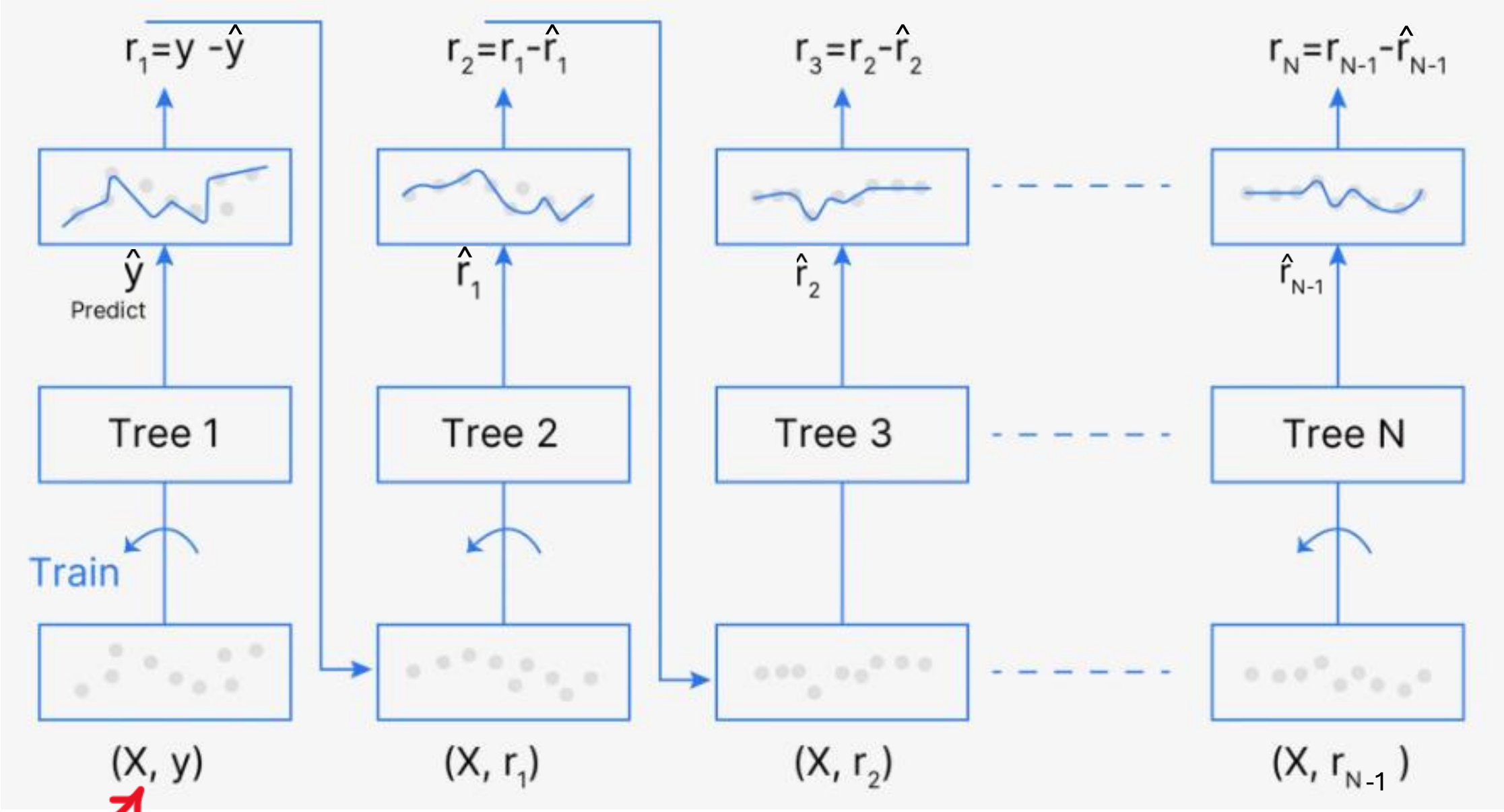
- **De ce funcționează:**
  *Fiecare model nou se concentrează pe **erorile anterioare**, transformând o serie de clasificatori slabi într-un model puternic și robust.*

- **Pe scurt:**
  AdaBoost = *învățare din greșeli prin reponderarea exemplelor dificile.*

Gradient Boosting steps:

- consider a regression problem where we use decision trees as predictors
- train a first *DecisionTreeRegressor* on the training set
- train a second *DecisionTreeRegressor* on the residual errors made by the first one
- train a third *DecisionTreeRegressor* on the residual errors made by the second one
- now we can make predictions on a new instance simply by adding up the predictions of all the 3 trees





original training set

- **Scopul:**

  Să combinăm mai mulți arbori de decizie simpli (slabi) astfel încât fiecare să corecteze greșelile celui anterior.

- **Ideea de bază:**

  În loc să antrenăm un singur model mare, antrenăm o serie de arbori care învață reziduurile (adică erorile rămase) de la pașii precedenți.

- **Pașii principali:**
  - Antrenăm primul arbore pe datele originale *(X, y)*.
  - Calculăm **reziduurile** = *valoarea reală – predicția arborelui*.
  - Antrenăm al doilea arbore să prezică **aceste reziduuri**.
  - Calculăm din nou reziduurile (ce nu a reușit să repare al doilea arbore).
  - Repetăm procesul pentru mai mulți arbori.
  - Predicția finală = suma tuturor predicțiilor (fiecare arbore contribuie puțin).

- **De ce funcționează:**

  Fiecare arbore învață unde a greșit sistemul anterior → modelul devine tot mai precis la fiecare iterație.

- **Pe scurt:**

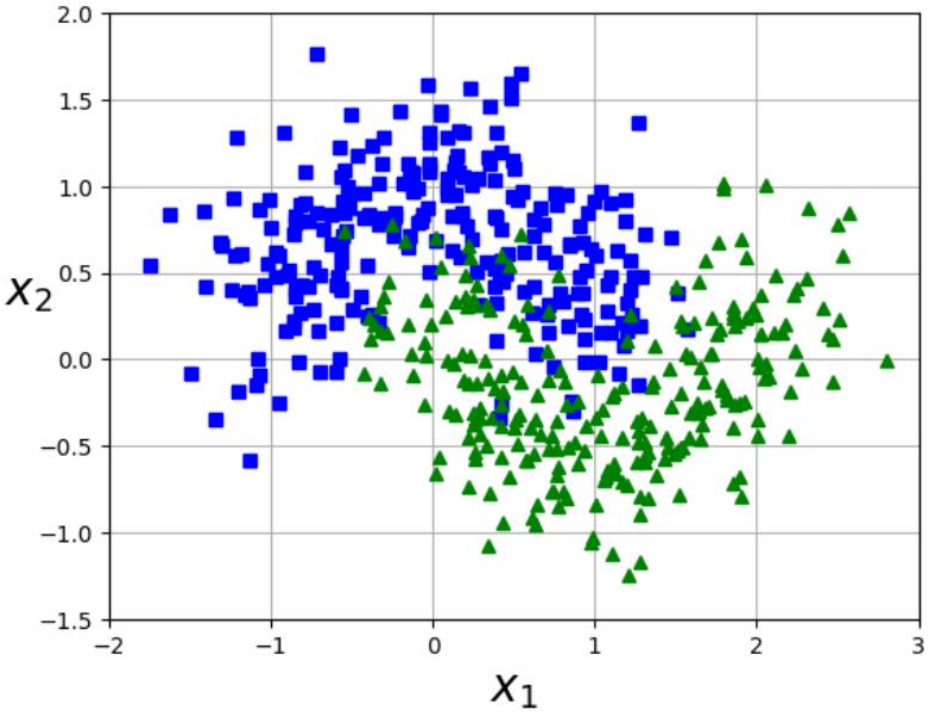
  Gradient Boosting = *învățare treptată din erori + combinarea arborilor pentru o predicție finală mai bună.*

Code part

AdaBoost

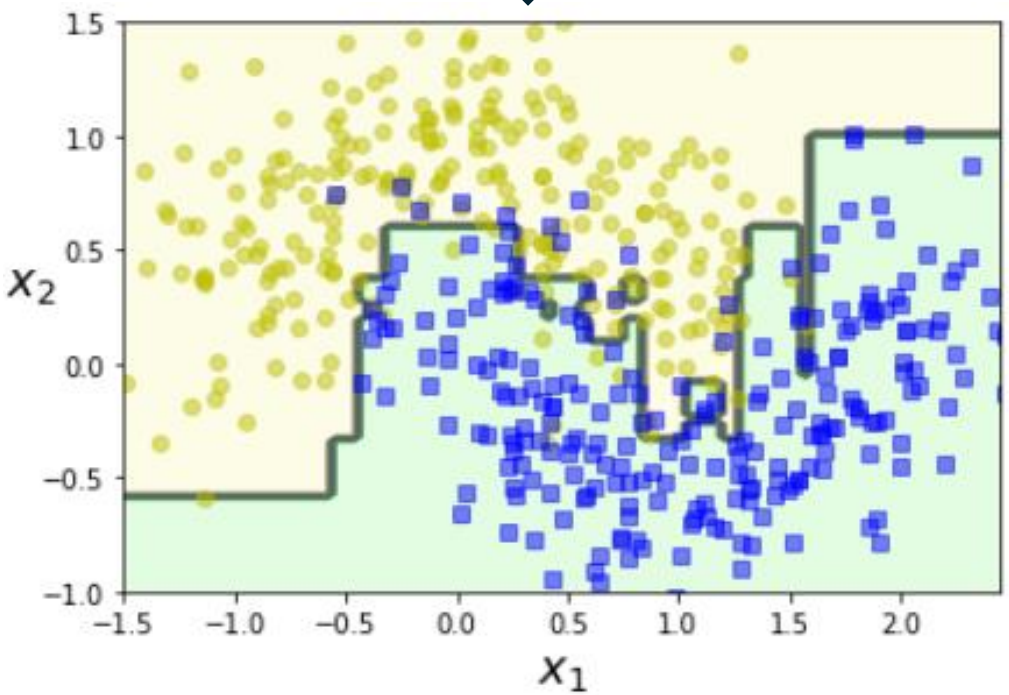
```python
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_moons

X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
```

plot_dataset() function from lab 3 ☺



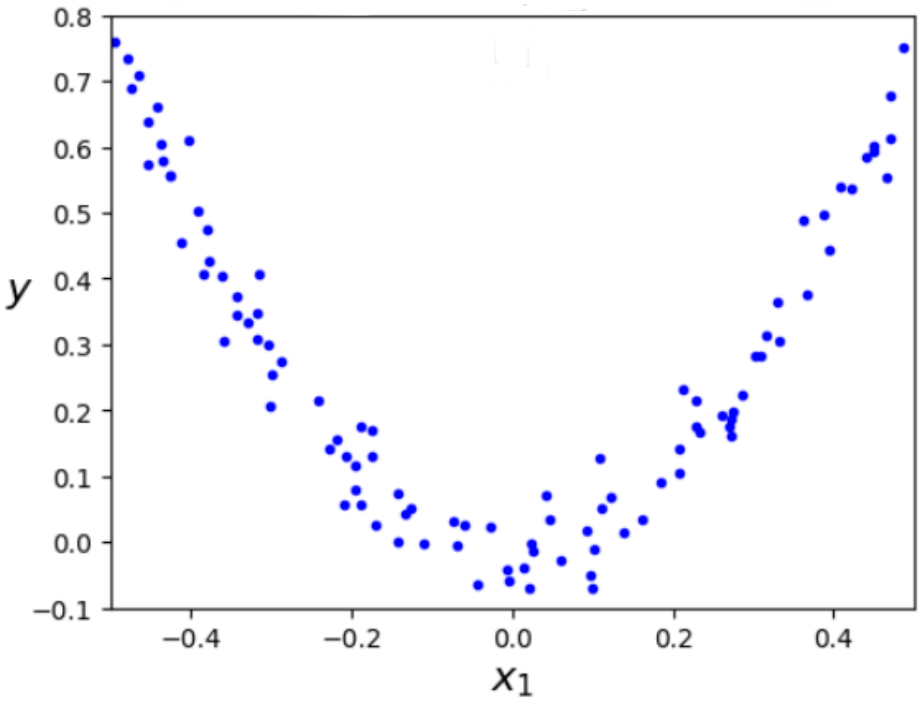
```python
from sklearn.ensemble import AdaBoostClassifier

ada_clf = AdaBoostClassifier(
    base_estimator=DecisionTreeClassifier(max_depth=1), n_estimators=200,
    learning_rate=0.5, random_state=42)
ada_clf.fit(X_train, y_train)
```

plot_decision_boundary()
function already done in lab



Gradient Boosting

```
np.random.seed(42)
X = np.random.rand(100, 1) - 0.5
y = 3*X[:, 0]**2 + 0.05 * np.random.randn(100)
```



train a first *DecisionTreeRegressor* on the training set

```
from sklearn.tree import DecisionTreeRegressor

tree_reg1 = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg1.fit(X, y)
```

1)

train a second *DecisionTreeRegressor* on the residual errors made by the first one

```
y2 = y - tree_reg1.predict(X)
tree_reg2 = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg2.fit(X, y2)
```

2)

train a third *DecisionTreeRegressor* on the residual errors made by the second one

```
y3 = y2 - tree_reg2.predict(X)
tree_reg3 = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg3.fit(X, y3)
```

3)

now we can make predictions on a new instance simply by adding up the predictions of all the 3 trees

```
X_new = np.array([[0.8]])
```

```
y_pred = sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3))
y_pred
```

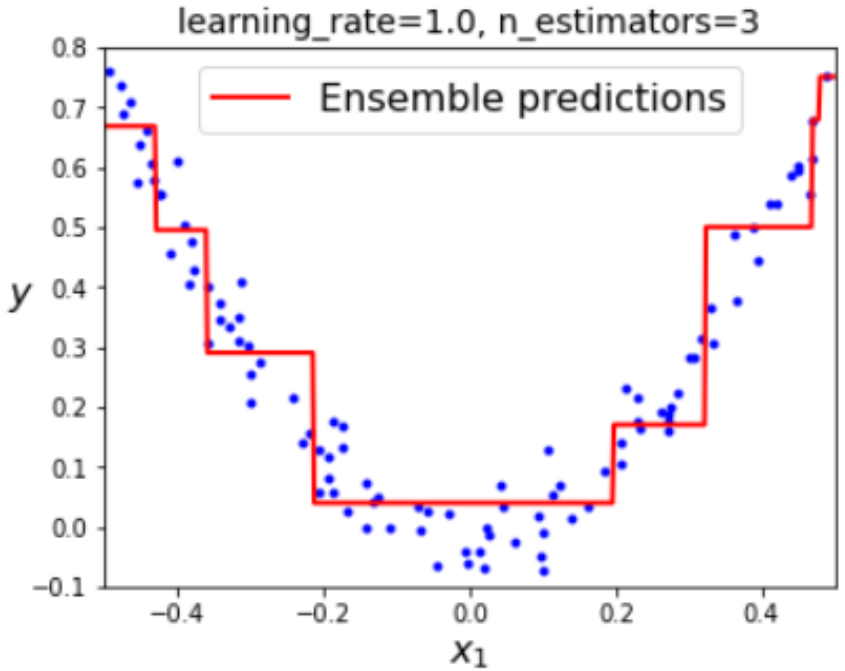
```
array([0.75026781])
```

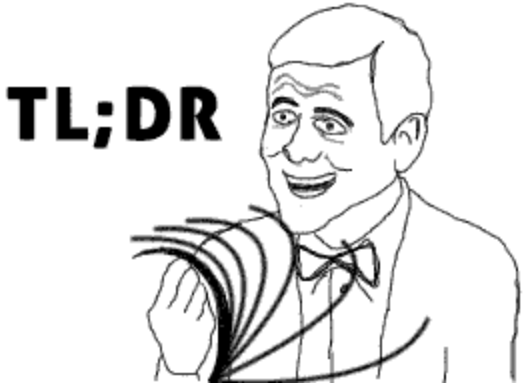
4)

...or in a simpler way...

```
from sklearn.ensemble import GradientBoostingRegressor

gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=3, learning_rate=1.0, random_state=42)
gbrt.fit(X, y)
```

plot_predictions()
function already done in lab

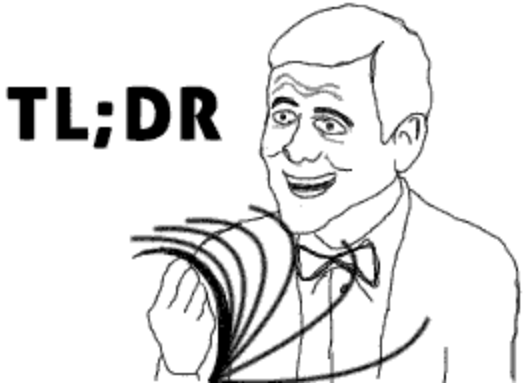




- **Decision trees:** learn by asking **if/else questions** to split data into **pure** regions.
  - **Classification:** predict the *majority class* in each leaf.
  - **Regression:** predict the *mean value* in each leaf.

- **Voting (Ensemble of Models):** combine multiple classifiers/regressors for better accuracy.
  - **Hard Voting:** majority vote of class predictions.
  - **Soft Voting:** average class probabilities and choose the highest.

- **Bagging and pasting:** train same model on **random subsets** of the data:
  - **Bagging:** *with replacement* (bootstrap sampling).
  - **Pasting:** *without replacement*.



- **Random Forests:** Ensemble of **Decision Trees** trained via **bagging**.

- **Boosting:** train models **sequentially**, each correcting previous errors.
  - **AdaBoost:** focus more on misclassified samples.
  - **Gradient Boosting:** fit new models to **residual errors**.

Exercises

Laboratory 4 – Exercises

1. Use the following regression dataset:

```
X = np.random.rand(350,1)
y = (X + 1.2)^3 - 10 + np.random.randn(350,1)/10,
```

in order to perform a regression task. Build a regression decision tree with `max_depth=3` and `min_samples_leaf=5`. Visualize the trained decision tree and plot the regression predictions.

2. Use the `digits` dataset to perform a classification task using bagging. Set the number of estimators to 200. The test split should be 15% of the initial dataset. Print the accuracy of the model.

3. Use the following regression dataset:

```
X = np.random.rand(160,1) - 1
y = 2*(X[:, 0])^2 + 0.2*np.random.randn(160),
```

in order to perform gradient boosting. The `GradientBoostingRegressor` class has the hyperparameters: `max_depth=2` and `n_estimators=100`. The test split is 25% of the initial dataset. Use early stopping to find the optimal number of trees and plot:

- ➢ the model's predictions with 100 estimators;
- ➢ the model's predictions with the optimal number of trees.

Solutions

1. Use the following regression dataset:

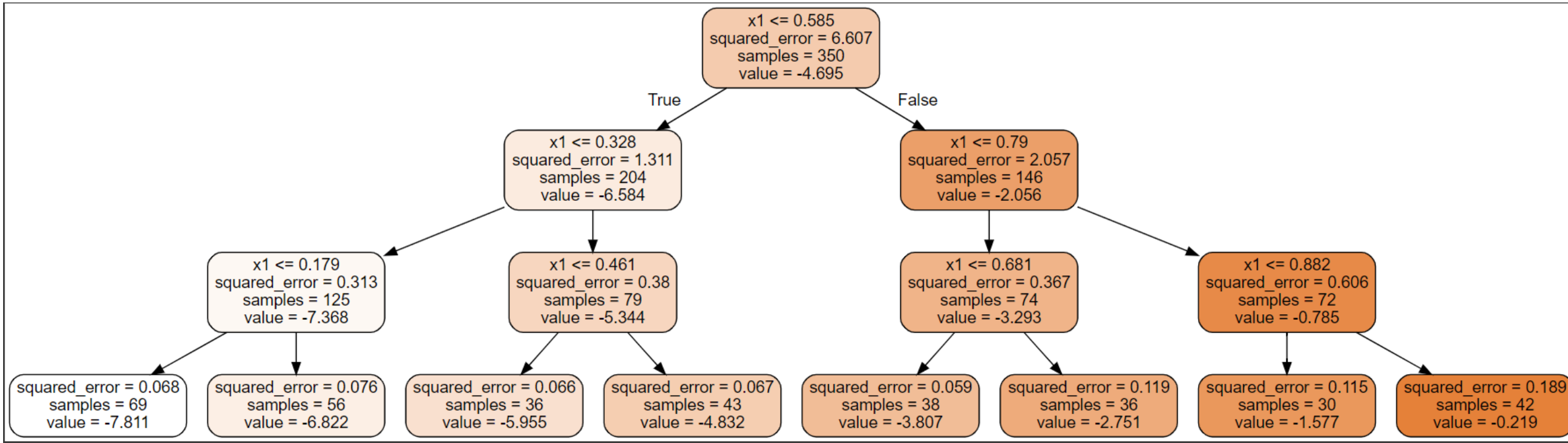
```
X = np.random.rand(350,1)
y = (X + 1.2)^3 - 10 + np.random.randn(350,1)/10,
```

in order to perform a regression task. Build a regression decision tree with `max_depth=3` and `min_samples_leaf=5`. Visualize the trained decision tree and plot the regression predictions.

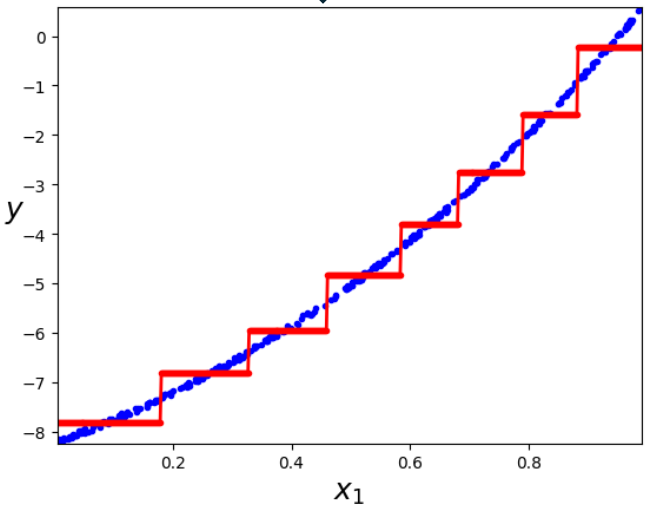
```python
import numpy as np

np.random.seed(42) # Set the random seed for reproducibility of results
X = np.random.rand(350, 1) # Generate the input data (X) using random values between 0 and 1, 350 samples
# Generate the output (y) based on a cubic transformation of X with some added noise
# The formula (X + 1.2)^3 - 10 creates a nonlinear relationship between X and y
# Noise is added by np.random.rand to make the regression task more realistic
y =(X + 1.2) ** 3 - 10 + np.random.rand(350, 1)/10
```

```python
from sklearn.tree import DecisionTreeRegressor

# Create an instance of DecisionTreeRegressor with a max depth of 3 and a minimum of 5 samples per leaf
# max_depth=3 restricts the complexity of the tree.
# min_samples_leaf=5 ensures each leaf has at least 5 samples, which helps smooth predictions.
tree_reg = DecisionTreeRegressor(max_depth=3, min_samples_leaf=5, random_state=42)

tree_reg.fit(X, y) # Fit the decision tree to the data
```

```python
import os
from graphviz import Source
from sklearn.tree import export_graphviz

# Export the decision tree in DOT format, which can be visualized using Graphviz.
export_graphviz(
        tree_reg,
        out_file=os.path.join(".", "regression_tree.dot"),
        feature_names=["x1"],
        rounded=True,
        filled=True
    )

Source.from_file(os.path.join(".", "regression_tree.dot"))
```



```python
import matplotlib.pyplot as plt

# Define a function to plot the decision tree predictions for regression.
# This function takes in the trained model, input data, and output data
# 'axes' define the plot boundaries based on the data.
def plot_regression_predictions(tree_reg, X, y, axes=[X[:,0].min(), X[:,0].max(), y.min(), y.max()]):
    # Generate a fine grid of points for plotting smooth predictions.
    x1 = np.linspace(axes[0], axes[1], 500).reshape(-1, 1)

    # Predict the output (y_pred) for these grid points using the trained model.
    y_pred = tree_reg.predict(x1)

    # Set the plot axis limits to the specified values in 'axes'
    plt.axis(axes)

    # Label the x-axis and y-axis with descriptive names
    plt.xlabel("$x_1$", fontsize=18)
    plt.ylabel("$y$", fontsize=18, rotation=0)

    # Plot the original data points (X, y) as blue dots
    plt.plot(X, y, "b.")

    # Plot the predicted values (x1, y_pred) as a red line with markers
    # This shows how the decision tree approximates the regression function.
    plt.plot(x1, y_pred, "r.-", linewidth=2, label=r"$\hat{y}$")

plot_regression_predictions(tree_reg, X, y)

plt.show()
```



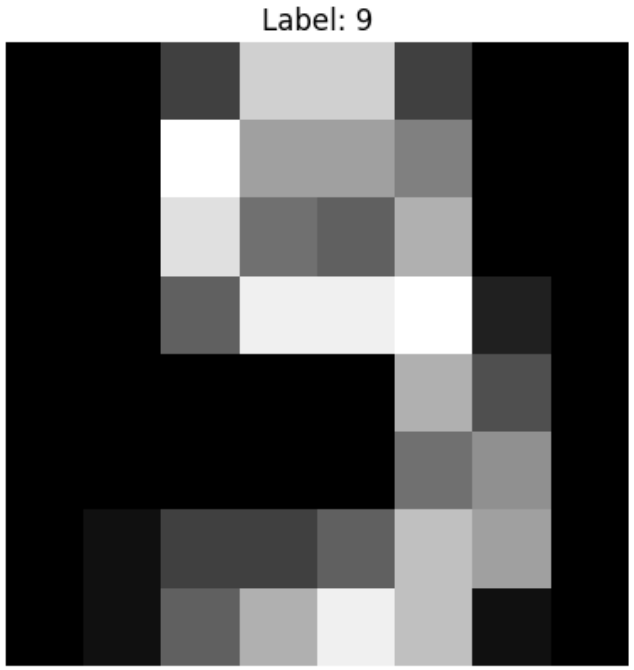
2. Use the `digits` dataset to perform a classification task using bagging. Set the number of estimators to 200. The test split should be 15% of the initial dataset. Print the accuracy of the model.

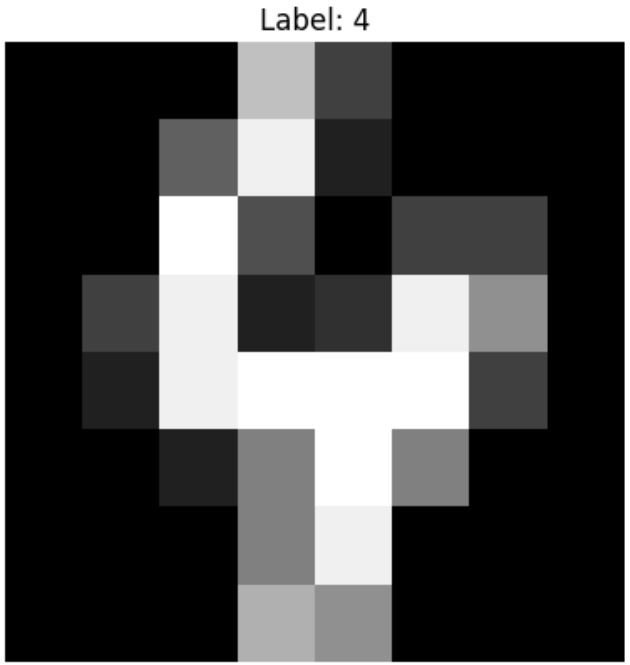
```
from sklearn import datasets
from sklearn.model_selection import train_test_split


# Load the digits dataset from sklearn's datasets module
# The digits dataset contains 8x8 pixel images of handwritten digits (0-9)
digits_dataset = datasets.load_digits()

# Split the dataset into training and testing sets
# 15% of the data is reserved for testing using test_size=0.15
# The random_state=0 ensures that the split is reproducible
X_train, X_test, y_train, y_test = train_test_split(digits_dataset['data'],digits_dataset['target'],test_size=0.15,random_state=0)
```

NOTE: this is how you can import and use digits dataset !

OPTIONAL: plotting some examples to understand the data you use ☺

```python
import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets

def show_random_sample(digits_dataset):
    # Choose a random index from the dataset
    random_index = np.random.randint(0, len(digits_dataset['images']))

    # Get the image and the corresponding label (digit)
    image = digits_dataset['images'][random_index]
    label = digits_dataset['target'][random_index]

    # Plot the image
    plt.imshow(image, cmap='gray')  # Display in grayscale
    plt.title(f"Label: {label}")    # Set the title to the corresponding label
    plt.axis('off')                 # Turn off axis for better visualization
    plt.show()

# Call the function to display a random sample
show_random_sample(digits_dataset)
```





```python
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

# Initialize the BaggingClassifier with a DecisionTreeClassifier as the base estimator
# n_estimators=200 means we are using 200 decision trees in the ensemble
# max_samples=100 specifies that each base classifier will be trained on 100 random samples
# bootstrap=True enables sampling with replacement (i.e., bootstrapping)
# random_state=42 ensures reproducibility
bag_clf = BaggingClassifier(
    DecisionTreeClassifier(), n_estimators=200,
    max_samples=100, bootstrap=True, random_state=42)

bag_clf.fit(X_train, y_train) # Fit the bagging classifier to the training data
y_pred = bag_clf.predict(X_test) # Use the trained classifier to predict the labels of the test data
```

```python
from sklearn.metrics import accuracy_score

# Print the accuracy of the model, which is the percentage of correct predictions on the test set
print(accuracy_score(y_test, y_pred))
```

```
0.9222222222222223
```

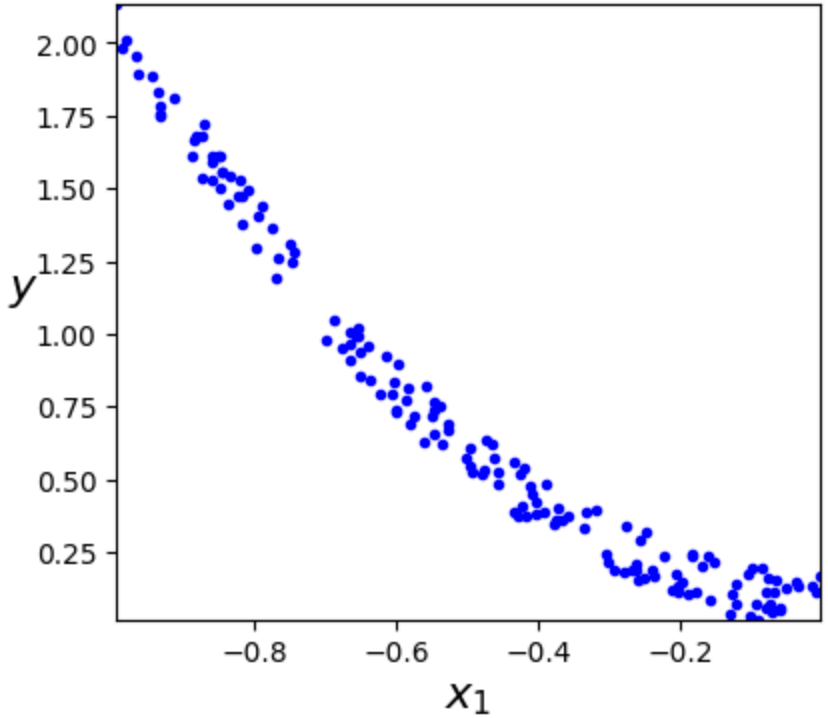
3. Use the following regression dataset:

```
X = np.random.rand(160,1) - 1
y = 2*(X[:, 0])^2 + 0.2*np.random.randn(160),
```

in order to perform gradient boosting. The `GradientBoostingRegressor` class has the hyperparameters: `max_depth=2` and `n_estimators=100`. The test split is 25% of the initial dataset. Use early stopping to find the optimal number of trees and plot:

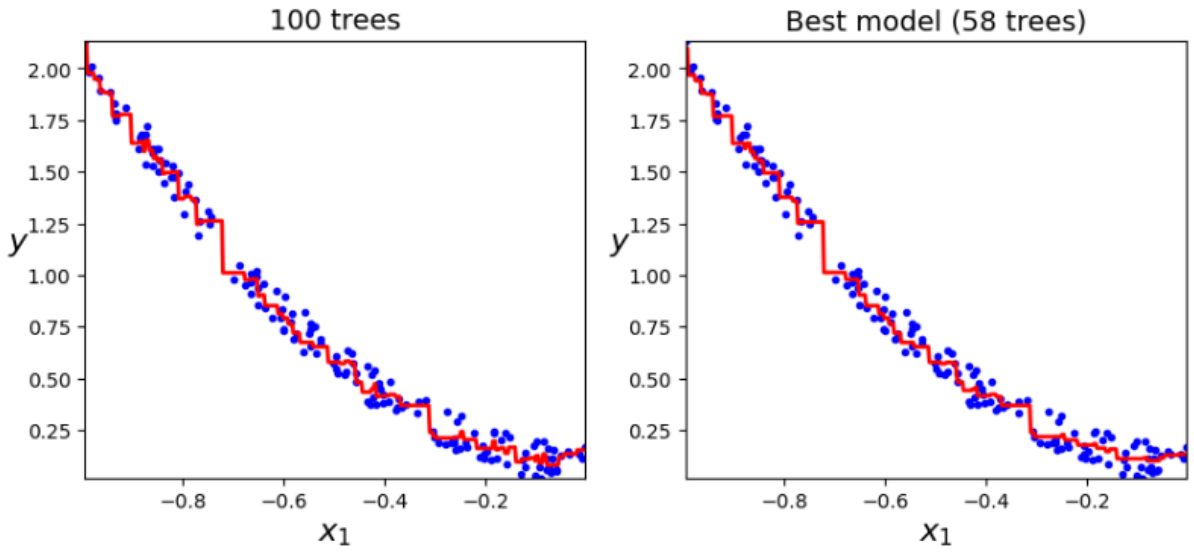
- the model's predictions with 100 estimators;
- the model's predictions with the optimal number of trees.

```python
from sklearn.metrics import mean_squared_error
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import train_test_split


# Create the dataset
# Generate X, a random sample of 160 points from a uniform distribution, shifted to the range [-1, 0]
X = np.random.rand(160, 1) - 1
# Generate y, the target variable based on a quadratic function with some added noise
y = 2*(X[:, 0])**2 + 0.2*np.random.rand(160)

# Split the data into a training set and a test set
# 25% of the data is used for test (test_size=0.25)
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.25, random_state=49)
```



```python
# Create a GradientBoostingRegressor model with max_depth=2 and 100 estimators (trees)
gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=100, random_state=42)
gbrt.fit(X_train, y_train) # Fit the model on the training data

# List comprehension to compute the mean squared error for each prediction (staged_predict)
# as the model adds more trees (one by one) in the gradient boosting process
errors = [mean_squared_error(y_val, y_pred)
          for y_pred in gbrt.staged_predict(X_val)]

# Find the optimal number of estimators (trees) that gives the lowest validation error
# np.argmin(errors) finds the index of the smallest error, which is our best model
# Add 1 because np.argmin gives zero-indexed output
bst_n_estimators = np.argmin(errors) + 1

# Create a new GradientBoostingRegressor with the optimal number of estimators
gbrt_best = GradientBoostingRegressor(max_depth=2, n_estimators=bst_n_estimators, random_state=42)
gbrt_best.fit(X_train, y_train) # Fit this optimized model on the training data
```

```python
# Function to plot the model's predictions
# Arguments:
# - regressors: the list of regressors to use for prediction
# - X, y: the original data
# - axes: the range for the x and y axes
# - label, style: the plot styles and labels for the data and predictions
def plot_predictions(regressors, X, y, axes, label=None, style="r-", data_style="b.", data_label=None):
    # Generate a fine grid of points in the x-axis range for smooth plotting
    x1 = np.linspace(axes[0], axes[1], 500)

    # Sum the predictions of all regressors (for an ensemble of regressors) over the x1 grid
    y_pred = sum(regressor.predict(x1.reshape(-1, 1)) for regressor in regressors)

    # Plot the original data points
    plt.plot(X[:, 0], y, data_style, label=data_label)

    # Plot the predicted values (smooth line based on the predictions)
    plt.plot(x1, y_pred, style, linewidth=2, label=label)

    # If a label is provided, add a legend to the plot
    if label or data_label:
        plt.legend(loc="upper center", fontsize=16)

    # Set the axis limits to the values specified in the 'axes' argument
    plt.axis(axes)

# Create a figure with two subplots
plt.figure(figsize=(10, 4))

# Plotting the model's predictions with 100 estimators
plt.subplot(121)  # Left subplot
plot_predictions([gbrt], X, y, axes=[X[:,0].min(), X[:,0].max(), y.min(), y.max()])
plt.title("100 trees", fontsize=14)
plt.ylabel("$y$", fontsize=16, rotation=0)
plt.xlabel("$x_1$", fontsize=16)

# Plotting the model's predictions with the optimal number of estimators (best model)
plt.subplot(122)  # Right subplot
plot_predictions([gbrt_best], X, y, axes=[X[:,0].min(), X[:,0].max(), y.min(), y.max()])
plt.title("Best model (%d trees)" % bst_n_estimators, fontsize=14)
plt.ylabel("$y$", fontsize=16, rotation=0)
plt.xlabel("$x_1$", fontsize=16)

# Display the plots
plt.show()
```



Thank you !